

Generic PVSS Framework with $O(1)$ Complexity Using CCA2-Secure Threshold Encryption

Liang Zhang^{*1}, Dongliang Cai^{*2}, Yiwen Gao², Haibin Kan^{†2}, Jiheng Zhang^{†1},
and Moti Yung³

scottzhang@ust.hk; 22110240060@m.fudan.edu.cn;
wgao21@m.fudan.edu.cn; hbkan@fudan.edu.cn; jiheng@ust.hk;
motiyung@gmail.com

*: Equal contributions

†: Corresponding authors

¹ The Hong Kong University of Science and Technology

² Fudan University

³ Google LLC and Columbia University

Abstract. Existing PVSS schemes suffer from at least $O(n)$ online complexity due to the need to individually encrypt and prove/ verify each of the n shares. In this work, we present a generic framework for constructing PVSS schemes with $O(1)$ complexity for share distribution and (the expected to be repeated numerous times) public verification. Our key insight lies in establishing a novel connection between PVSS and CCA2-Secure threshold encryption (CCATE), which enables public verifiability enforced by Non-Interactive Zero-Knowledge (NIZK) proofs. We show that a CCATE scheme can be generically transformed into a secure PVSS scheme, eliminating the $O(n)$ bottleneck per on-line operations. We instantiate the framework by presenting two CCATE constructions: 1) A pairing-free scheme based on a committee-based Distributed Key Generation (DKG) protocol and Threshold ElGamal encryption. 2) A silent setup scheme leveraging a non-interactive distributed key generation, relying on Power-of-Tau ceremony. Furthermore, we introduce solutions for dynamic membership updates in both DKG constructions, demonstrating their practicality and adaptability for real-world applications. The scheme is based on an off-line setup stage (before a specific value to share is given) where the $O(n)$ complexity is dealt with. Although our schemes incur higher setup costs, they drastically reduce the complexity of the critical distribution and verification stages to constant time. This trade-off marks a significant advancement in the scalability of PVSS-based systems, especially in the context of blockchain modern transactions. Conceptually, the work points out how variants of the notion of Threshold Encryption can potentially serve as a “compression mechanism” for information sharing schemes.

Keywords: PVSS · CCA2-Secure Threshold Encryption · Public Verifiability · Zero-Knowledge Proofs · Scalable Cryptography

1 Introduction

Scaling decentralized systems or blockchains presents a fundamental challenge: as the number of participants increases, the overhead of most decentralized protocols tends to grow linearly. This is an inherent consequence of the distribution of responsibilities and data among multiple nodes. This situation has made us wonder whether a secret sharing scheme (which is often used in distributed systems) can achieve $O(1)$ distribution and verification complexity while still distributing n shares. This may sound paradoxical (involving n shares based on a constant value), which, information theoretically speaking, is even provable, yet we attack this problem directly and computationally.

In fact, we positively answer the question by concretely dealing with “publicly verifiable secret sharing” (PVSS) scheme [1] as a critical example. We propose a novel paradigm by shifting expensive operations to a one-time setup stage, thus enabling constant-time complexity for both distribution and verification (verification is an operation which can be repeated independently by many). Both the PVSS scheme and the threshold encryption scheme (which yields constant-size ciphertext) share a common requirement: support for threshold decryption or recovery. Building on this insight and employing threshold encryption, we explore a generic framework to construct PVSS schemes that achieve $O(1)$ -complexity in both distribution and verification.

To demonstrate the practicality of our approach, we present two concrete implementations: one based on Threshold ElGamal Encryption [25,38], and the other utilizing the silent setup-based threshold encryption scheme [28]. These constructions significantly outperform existing (old and recent) PVSS schemes [2,4,7,8,9,10], which incur at least $O(n)$ online complexity.

PVSS can be seamlessly integrated into blockchain systems where setup and distribution data are publicly stored on-chain. Verification of shares can then be automatically executed by smart contracts (which offline read the setup data), ensuring correctness without manual intervention. Further, the decoupling of share distribution from secret reconstruction makes PVSS a powerful commitment scheme [11], while allowing threshold (fault-tolerant) opening. Therefore, PVSS is vital for decentralized applications such as e-voting [2,4], on-chain data management [3], secure multiparty computation (MPC) [10,5], witness encryption [6], distributed randomness beacons [7,8,13,14,15], proof-of-stake (PoS) consensus [16] and DAOs [17].

1.1 Related Works

The concept of Publicly Verifiable Secret Sharing (PVSS) originated with Stadler’s pioneering work in 1996 [1]. Building on this foundation, Schoenmakers [2] proposed a PVSS scheme based on ElGamal encryption, featuring a verification complexity of $O(nt)$. Recently, several $O(n)$ -complexity PVSS schemes have emerged [4,7,8,10]. Among them, SCRAPE [7] stands out for leveraging Reed–Solomon codes to prove that shares originate from a common secret. ALBA-TROSS [8] builds on SCRAPE by adopting packed Shamir secret sharing [12] and

introducing a low degree exponent interpolation proof. In YOLO YOSO [9], two PVSS protocols—HEPVSS and DHPVSS—are introduced. HEPVSS employs a modified ElGamal scheme with additive homomorphism to enable efficient validity proofs, while DHPVSS achieves constant size proofs (two field elements) using a single DLEQ proof to verify the polynomial degree, though the sharing phase still incurs $O(n)$ complexity. Extending DHPVSS, qCLPVSS [10] enables secret recovery over $\mathbb{Z}_p$, making it suitable for distributed key generation (DKG) and YOSO-style MPC, with its security grounded in class group assumptions. More recently, Baghery et al. [4] introduced the notion of Preconstructed PVSS, recognizing Schoenmakers’s scheme as a special case and proposing a more efficient variant inspired by SCRAPE. Zhang et al. [18] presented a novel PVSS scheme using the Sigma protocol [21] and Fiat–Shamir heuristic [22] to generate non-interactive zero-knowledge (NIZK) proofs for encrypted shares, replacing Reed–Solomon codes with Lagrange interpolation to establish binding among all encrypted shares.

1.2 Contributions

We conceptually show the power of Threshold Encryption variants to “compress” the online stage of distribution-oriented primitives. Concretely for PVSS:

1. **CCATE Using NIZK Proofs:** We formalize CCA2-secure Threshold Encryption (CCATE) schemes that incorporate built-in public verifiability through the use of NIZK proofs. CCATE extends standard threshold encryption by requiring that all critical operations be publicly verifiable. Specifically:
 - Setup Verification: The setup implements a distributed key generation (DKG) protocol [19,20], ensuring that the master public key is correctly formed from honest contributions.
 - Ciphertext Verification: To encrypt a secret s , the dealer produces a ciphertext C along with a NIZK proof π_s , demonstrating that C is well formed and contains a valid plaintext under the master public key. This proof is generated using the Sigma protocol and the Fiat–Shamir heuristic.
 - Partial Decryption Verification: Each shareholder must accompany their partial decryption share D_i with a NIZK proof π_{sk_i} , typically realized as a Discrete Logarithm Equality (DLEQ) proof [24]. This ensures that D_i was correctly computed using the secret key sk_i .
2. **Generic $O(1)$ Complexity PVSS Framework From CCATE:** We present a generic construction that shows how a CCATE scheme can be used to build a PVSS scheme. As shown in Figure 1, the PVSS algorithms (PVSS.Setup, PVSS.Share, PVSS.Verify, etc.) are constructed by directly invoking the corresponding algorithms of the underlying CCATE scheme (**Setup**, **Encrypt**, **VerifyCT**, etc.). This framework elegantly reduces the problem of building an efficient PVSS to the problem of building an efficient CCATE scheme. We prove that the security properties of such PVSS schemes (correctness, public verifiability, and IND1-Secrecy) are directly implied by the CCATE scheme.

3. **Two $O(1)$ PVSS Constructions:** To instantiate our framework, we introduce two concrete CCATE-based constructions, yielding the first PVSS schemes with $O(1)$ complexity during the distribution and verification phases:
 - Construction I (Pairing-Free): This design utilizes a committee-based DKG protocol and a Threshold ElGamal encryption scheme [25], avoiding the overhead of bilinear pairings.
 - Construction II (Silent Setup): This approach adopts the "silent setup" paradigm [27,28], in which parties generate their partial key pairs non-interactively, relying solely on a Power-of-Tau ceremony [26].

1.3 Applications

OnChain Data Management: PVSS is a foundational technology for On-Chain Data Management, elevating blockchains from basic transaction ledgers to secure platforms for handling private data [29,30]. It enables confidentiality, integrity, and accountability in trustless environments, as demonstrated by systems like Calypso [3] and Cassiopeia [6]. Rather than encrypting large datasets directly, PVSS is used to manage a symmetric encryption key k for the data. The data owner (writer) encrypts their secret m with k , and then distributes k among a committee of trustees via PVSS. The encrypted shares and accompanying NIZK proofs are published on-chain as part of the write transaction.

Secure Multi-Party Computation (SMPC): PVSS serves as a trustless mechanism for ensuring the correct and consistent distribution of secret shares, enabling full auditability by any network participant. In e-voting systems [2,4], PVSS is employed to distribute decryption shares of encrypted ballots among election authorities. Its public verifiability allows voters and observers to confirm the integrity of the process, enhancing transparency and trust without compromising ballot secrecy. For Mempool privacy [31,32], PVSS schemes with $O(1)$ complexity can be used to commit transactions in a concealed manner, preventing miners from viewing their contents and mitigating information-driven bias in transaction selection. These transactions remain recoverable through coordinated decryption by PVSS shareholders. Additional SMPC applications include MPC wallets [5] and PoS consensus protocols [16].

DRB with High Throughput: PVSS is fundamental to achieving fully publicly verifiable distributed randomness beacon (DRB) protocols [7,8,13,14,15,18]. Notably, Zhang et al. [18] pioneered a general asynchronous beacon framework grounded in PVSS. Our proposed $O(1)$ -complexity instantiations would significantly reduce communication overhead and computational latency, leading to substantial improvements in throughput, bandwidth efficiency, and overall scalability. Furthermore, the support for dynamicity of parties, enabled through either committee-based DKG or silent setup, offers a robust mechanism for seamless participant reconfiguration. This flexibility significantly improves the adaptability and practicality of randomness beacon protocols in decentralized real-world environments.

1.4 Organization

Section 2 introduces the cryptographic primitives and critical concepts essential to our work. Section 3 presents the core framework for constructing PVSS schemes with $O(1)$ distribution and verification complexity, derived generically from a CCATE scheme. In Section 4, we instantiate this framework using a pairing-free PVSS construction based on a committee-based DKG setup and Threshold ElGamal encryption. Section 5 then provides a second instantiation, building on the silent setup paradigm. Section 6, in turn, evaluates the setup costs of both approaches and compares their computational and communication complexities with state-of-the-art PVSS schemes. Section 7 concludes the paper and discusses broader implications for scalable distributed systems. The experimental results, presented in Appendix E, confirm the constant-size complexity advantage of the proposed PVSS scheme in both the distribution and verification phases.

2 Preliminaries

Let $\mathbb{G}$ be a cyclic group of prime order p with generator g , and let $\mathbb{Z}_p$ denote the finite field of integers modulo p . Define $\mathcal{PK} \subseteq \mathbb{Z}_p \times \mathbb{G}$ as the set of public-key pairs (s, g^s) , where $s \in \mathbb{Z}_p$.

2.1 Polynomials over $\mathbb{Z}_p$

Secret Sharing Given a secret $s \in \mathbb{Z}_p$ (where p is prime), construct a polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i \pmod{p}$, where $a_i \in \mathbb{Z}_p$ are random coefficients. Shares are points $(x_i, f(x_i))$ for $x_i \neq 0$. It ensures two key properties.

- Robustness: Any t or more shares can reconstruct s ;
- Perfect secrecy: Fewer than t shares reveal no information about s .

Lagrange Interpolation (on $\mathbb{G}$) To reconstruct $s = f(0)$ from at least t shares, consider the set $\{(x_j, y_j)\}_{j=1}^t$ without loss of generality, one computes $s = \sum_{j=1}^t y_j \cdot L_j(0)$, where $L_j(x) = \prod_{\substack{1 \leq k \leq t \\ k \neq j}} \frac{x - x_k}{x_j - x_k}$ is the Lagrange basis polynomial. Similarly, given t shares $\{(x_j, g^{y_j})\}$, one can uniquely interpolate g^s . Denote **Intpl** as the Lagrange interpolation algorithm. The following properties can be implied:

- Homomorphism: $\text{Intpl}(\{(j, \sum_j x_j)\}) = \sum \text{Intpl}(\{(j, x_j)\})$, due to homomorphism of polynomials.
- Uniqueness: As any t shares can correctly recover s , implying uniqueness of the reconstruction process.

2.2 Sigma protocol and NIZK Proofs

A Sigma protocol is a 3-move interactive proof system for an NP relation $R = \{(x, y)\}$, where y is a statement and x is the witness. The basic procedure is as follows:

1. The prover P sends a commitment a for the statement to the verifier V .
2. The verifier V sends a random value $c \in \mathcal{C}$ back to the prover P .
3. The prover P sends a response z , calculated based on (y, a, c) , to the verifier V , who decides whether to accept the statement.

The transcript, i.e., (a, c, z) , is called the commitment, challenge, and response, respectively. A Sigma protocol has the following properties:

- **Completeness:** The honest verifier always accepts if the prover is honest. Formally, for every $(x, y) \in \mathcal{R}$, $\Pr[P(x, y) \leftrightarrow V(y) = \text{accept}] = 1$.
- **Soundness:** There exists an efficient extractor that given two accepting transcripts (a, c, z) and (a, c', z') for y such that $c \neq c'$, outputs a witness x such that $(x, y) \in \mathcal{R}$.
- **Zero-Knowledge:** There exists an efficient simulator that on input (y, c) , where y is a statement and $c \in \mathcal{C}$ is a challenge, can output a transcript (a, c, z) indistinguishable from real protocol executions. Formally, $\{Sim(y, c) : c \xleftarrow{\$} \mathcal{C}\} \approx_c \{\mathcal{V}((P(x, y) \leftrightarrow V(y)))\}$.

Fiat-Shamir heuristic [22] enables one to convert interactive Sigma protocols to NIZK proofs by replacing the verifier's challenge with $c = H(y, a)$, where H is a random oracle. The NIZK proofs for statement y is denoted as $\pi_x = (a, c, z)$.

2.3 Publicly Verifiable Secret Sharing (PVSS)

Definition 1 (PVSS). A PVSS scheme includes six probabilistic polynomial-time (PPT) algorithms (PVSS.Setup, PVSS.Share, PVSS.Verify, PVSS.PreRecon, PVSS.DecShareVrf, PVSS.Recon):

1. $(\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \text{PVSS.Setup}(\lambda, t, n)$: The algorithm outputs public parameters pp , containing at least a master public key pk and each party obtains a key pair $(\text{pk}_i, \text{sk}_i)$.
2. $\text{bool} \leftarrow \text{PVSS.SetupVrf}(\text{pp}, \{\text{pk}_i\}_{i \in [1, n]})$: Inputs the secret keys and the public parameters pp , verify whether the secret keys can interpolate a unique value that equals $\text{pk} \in \text{pp}$.
3. $(C, \pi_s) \leftarrow \text{PVSS.Share}(s, \text{pp})$: Input a secret $s \in \mathbb{Z}_p$ and public parameters pp , the dealer calculates the encrypted shares, embedded in C , and generates a non-interactive zero-knowledge (NIZK) proof π_s to prove the validity of all shares.
4. $\text{bool} \leftarrow \text{PVSS.Verify}(C, \pi_s)$: Any external verifier can check whether the shares are correctly embedded in the given (C, π_s) .
5. $D_i \leftarrow \text{PVSS.PreRecon}(C, \text{sk}_i)$: Each shareholder privately decrypts C using secret key sk_i to obtain decrypted share D_i .

6. $bool \leftarrow \text{PVSS.DecShareVrf}((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)$ Anyone receive a decrypted D_i can check whether it is correctly calculated using proof π_{sk_i} .
7. $S \leftarrow \text{PVSS.Recon}(C, \{D_i\}_{i \in Q})$: Given the encrypted shares C and t corrected decrypted shares $\{D_i\}$, it outputs the secret S , such that $(s, S) \in \mathcal{PK}$.

A PVSS scheme satisfies the properties of **correctness**, **public verifiability**, and **IND1-Secrecy**.

- **Correctness** If the dealer and shareholders follow the protocol, then any t decrypted shares can reconstruct the secret S , such that $(s, S) \in \mathcal{PK}$.

$$\Pr \left[\begin{array}{l} (\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \text{PVSS.Setup}(\lambda, t, n), \\ (C, \pi_s) \leftarrow \text{PVSS.Share}(s, \text{pp}), \\ \text{true} \equiv \text{PVSS.Verify}(C, \pi_s) \wedge \\ \{D_i \leftarrow \text{PVSS.PreRecon}(C, \text{sk}_i)\}_{i \in [1, n]}, \\ \{\text{true} \equiv \text{PVSS.DecShareVrf}((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)\} \end{array} : \begin{array}{l} \forall Q \subseteq [1, n], |Q| \geq t : \\ S \leftarrow \text{PVSS.Recon}(C, \{D_i\}_Q) \wedge \\ S \equiv g^s \end{array} \right] = 1.$$

- **Public Verifiability** Public verifiability allows anyone to verify the correctness of the encrypted shares from the dealer. Besides, the decrypted shares should also be publicly verifiable in the reconstruction phase. Let $\mathcal{A}$ denote a PPT adversary in the definitions below, and let $\text{negl}(\lambda)$ represent a negligible probability.

- **Verifiability of Setup**

$$\Pr \left[\begin{array}{l} (\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \mathcal{A}(\lambda, t, n) \wedge \\ ((\text{sk}_i, \text{pk}_i) \notin \mathcal{PK}^4 \vee \text{Intpl}(\{(j, \text{sk}_j)\}) \equiv \perp) \end{array} : \text{true} \equiv \text{PVSS.SetupVrf}(\text{pp}, \{\text{pk}_i\}) \right] \leq \text{negl}(\lambda).$$

- **Verifiability of Distribution**

$$\Pr \left[\begin{array}{l} (\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \text{PVSS.Setup}(\lambda, t, n), \\ (C, \pi_s) \leftarrow \mathcal{A}(s, \text{pp}, \{\text{pk}_i\}) \end{array} : \begin{array}{l} \text{true} \equiv \text{PVSS.Verify}(C, \pi_s) \wedge \\ \nexists s \text{ s.t. } (C, \cdot) \equiv \text{PVSS.Share}(s, \text{pp}) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Verifiability of Decryption**

$$\Pr \left[\begin{array}{l} (\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \text{PVSS.Setup} \\ (\lambda, t, n); (C, \{D_i\}, s) \leftarrow \mathcal{A}(\text{pp}), \end{array} : \begin{array}{l} \{\text{true} \equiv \text{PVSS.DecShareVrf} \\ ((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)\}_{i \in [1, n]} \wedge \\ g^s \equiv \text{PVSS.Recon}(C, \{D_i\}) \end{array} \right] \leq \text{negl}(\lambda).$$

- **IND1-Secrecy** IND1-Secrecy guarantees that an adversary, given all public parameters and corrupting $t - 1$ secret keys $\{\text{sk}_i\}$, can distinguish the secret S from a random value with non-negligible advantage in the following game against a challenger.

⁴ In the silent setup-based approach, pk_i contains additional information to check its correctness, refer to the **VerifySetup** algorithm in Figure 4.

1. The challenger initializes the system by executing the PVSS.Setup algorithm, and provides all public parameters to $\mathcal{A}$. It sends key pairs for uncorrupted parties and sends the public keys to the adversary.
2. $\mathcal{A}$ generates secret keys for the corrupted parties and submits the corresponding public keys to the challenger.
3. The challenger selects two random secrets x_0 and x_1 from the secret space, and flips a fair coin $b \xleftarrow{\$} \{0, 1\}$. It then runs the PVSS.Share algorithm using x_0 as the secret and delivers to $\mathcal{A}$ all public outputs of this phase, along with the value x_b .
4. Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$.

The advantage of $\mathcal{A}$ is defined as $|\Pr[b = b'] - \frac{1}{2}|$.

3 A Generic PVSS Framework from CCA2-Secure Threshold Encryption (CCATE)

Public-key encryption schemes that are IND-CPA secure under classical hardness assumptions can be upgraded to IND-CCA2 security using well-established generic transformations. One key approach is Encrypt-then-MAC [33]. It combines any CPA-secure encryption with a strongly unforgeable message authentication code (MAC). Any ciphertext modification will fail MAC verification, blocking access to the decryption oracle. Another widely used method is the Fujisaki–Okamoto transform [34]. In the random oracle model, it derives encryption randomness and symmetric keys via hash functions. During decryption, malformed ciphertexts are rejected in a reconciliation step. A third category includes structure-preserving schemes like Cramer–Shoup encryption [35] or NIZK proofs [36]. It embeds algebraic integrity checks directly into the ciphertext. In this paper, we use NIZK proofs so as to enable publicly verifiable tests for ciphertext well-formedness.

3.1 CCATE Definition and Security Properties

A CCA2-secure threshold encryption (CCATE) scheme, adapted from [23], is defined by the following algorithms. This paper focuses on CCATE schemes instantiated over elliptic curve groups.

1. $(\text{pp}, \{sk_i, pk_i\}) \leftarrow \text{Setup}(\lambda, t, n)$: given a security parameter λ and integers $t, n \in \mathbb{Z}_p$ (with $1 \leq t \leq n$) denoting the number of decryption parties n and the decryption threshold t , this algorithm outputs $(\text{sk}, \text{pk}, \{sk_i, pk_i\})$, where (sk, pk) is the master key pair, $\{sk_i, pk_i\}$ is party P_i 's key pair. Denote g as a random generator of group $\mathbb{G}$. Note that sk is not required to be recovered in this paper. Denote the public parameters $\text{pp} \leftarrow (\mathbb{Z}_p, \mathbb{G}, g, t, n, \text{pk}, \{pk_i\})$.
2. $\text{bool} \leftarrow \text{VerifySetup}(\text{pp}, \{pk_i\})$: given public parameters pp and secret keys $\{sk_i\}$, check whether the secret keys can uniquely interpolate the master key $\text{pk} \in \text{pp}$.

3. $(C, \pi_s) \leftarrow \mathbf{Encrypt}(s, \mathbf{pp})$: given public parameters $\mathbf{pp}$ and a secret s , outputs a ciphertext C using S as plaintext, where $(s, S) \in \mathcal{PK}$. The process is a randomized algorithm.
4. $\text{bool} \leftarrow \mathbf{VerifyCT}(C, \pi_s)$: input a constant-size ciphertext C and the corresponding non-zero knowledge proofs π_s , it outputs 1 if C is considered valid and 0 otherwise.
5. $D_i \leftarrow \mathbf{PartialDec}(C, \mathbf{sk}_i)$: input th ciphertext C and P_i 's private key $\mathbf{sk}_i$, it outputs a partial decrypted value D_i if $\mathbf{VerifyCT}(C, \pi_s)$ returns 1, otherwise it returns $\perp$.
6. $\text{bool} \leftarrow \mathbf{VerifyPD}((D_i, \pi_{\mathbf{sk}_i}), \mathbf{pk}_i, C)$: takes in P_i 's decrypted value D_i , public key $\mathbf{pk}_i$ and the ciphertext C , it outputs either 1 or 0. If 1, D_i is said to be a valid decryption share; otherwise, D_i is invalid and is denoted as $\perp$.
7. $S \leftarrow \mathbf{Decrypt}(C, \{D_i\}_{i \in Q})$: given the ciphertext C and a subset $Q \subset \{1, \dots, n\}$ of size $t = |Q|$ with valid decryption shares $\{D_i\}_{i \in Q}$, this algorithm outputs a plaintext $S = g^s$. If Q contains $\perp$, then $S \leftarrow \perp$.

The security properties of CCATE scheme are as follows.

Definition 2 (Correctness). *Let $\mathcal{S}$ be the message space. For all $s \in \mathcal{S}$, $\mathbf{pp}, \{\mathbf{pk}_i, \mathbf{sk}_i\} \leftarrow \mathbf{Setup}(\mathbf{p})$, $\mathbf{VerifySetup}(\mathbf{pp}, \{\mathbf{pk}_i\}) \equiv \text{true}$, for any ciphertext $(C, \pi_s) \leftarrow \mathbf{Encrypt}(s, \mathbf{pp})$, $\mathbf{VerifyCT}(C, \pi_s) \equiv \text{true}$ and any set $Q \subseteq [1, n], |Q| \geq t$, we have the following:*

$$\Pr[(s, \mathbf{Decrypt}(C, \{\mathbf{PartialDec}(C, \mathbf{sk}_i)\}_{i \in Q})) \in \mathcal{PK}] = 1.$$

Definition 3 (Robustness). *For a message s , $\mathbf{pp}, \{\mathbf{pk}_i, \mathbf{sk}_i\} \leftarrow \mathbf{Setup}(\mathbf{p})$, $\mathbf{VerifySetup}(\mathbf{pp}, \{\mathbf{pk}_i\}) \equiv \text{true}$, $(C, \pi_s) \leftarrow \mathbf{Encrypt}(s, \mathbf{pp})$, $\mathbf{VerifyCT}(C, \pi_s) \equiv \text{true}$ and two sets $Q, Q' \subseteq [1, n], |Q| = |Q'| \geq t, Q \neq Q'$, we have the following:*

$$\Pr[\mathbf{Decrypt}(C, \mathbf{PartialDec}(C, \{\mathbf{sk}_i\}_{i \in Q})) \neq \mathbf{Decrypt}(C, \mathbf{PartialDec}(C, \{\mathbf{sk}_i\}_{i \in Q'}))] < \text{negl}(\lambda).$$

Definition 4 (Public Verifiability). *The outputs of $\mathbf{Setup}$, $\mathbf{Encrypt}$ and $\mathbf{PartialDec}$ are publicly verifiable.*

PV1 *The process of generation of $(\mathbf{pp}, \{\mathbf{pk}_i, \mathbf{sk}_i\})$ can be verified publicly.*

PV2 *A ciphertext (C, π_s) can be publicly checked via $\mathbf{VerifyCT}(C, \pi_s) \stackrel{?}{=} \text{true}$. This property implies that C is syntactically valid or decryptable given at least decrypted shares.*

PV3 *A decrypted share D_i from valid ciphertext C can be checked using $\mathbf{VerifyPD}((D_i, \pi_{\mathbf{sk}_i}), \mathbf{pk}_i, C) \stackrel{?}{=} \text{true}$.*

Definition 5 (Indistinguishability under chosen-ciphertext attack (IND-CCA2 or CCA2)). *A non-interactive (t, n) -threshold encryption scheme is said to be secure against chosen-ciphertext attack and adaptive corruptions if no PPT adversary can achieve a non-negligible advantage in the following security game:*

1. *The challenger runs $\mathbf{Setup}(\lambda, t, n)$ to obtain public parameters $\mathbf{pp}$, a vector of key pairs $\{\mathbf{sk}_i, \mathbf{pk}_i\}$ for each party P_i . It publishes $\mathbf{pp}$ and $\{\mathbf{pk}_i\}$ to the adversary $\mathcal{A}$ and keeps $\{\mathbf{sk}_i\}$ privately.*

2. The adversary $\mathcal{A}$ adaptively issues queries of the following types:
 - *Corruption query*: $\mathcal{A}$ queries sk_i to obtain no more than $t - 1$ distinct private keys.
 - *Encryption & Decryption query*: The adversary $\mathcal{A}$ queries the encryption oracles on arbitrary valid plaintext s to obtain ciphertext C . Further, the challenger sends $D_i = \text{PartialDec}(C, \{\text{sk}_i\})$ for $i \in [1, n]$ to $\mathcal{A}$.
3. The challenger selects two messages s_0 and s_1 of equal length and calculates a challenge ciphertext $C^* = \text{Encrypt}(s_0, \text{pp})$. It sends (s_b, C^*) to $\mathcal{A}$, where $b \xleftarrow{\$} [0, 1]$.
4. $\mathcal{A}$ issues additional queries as in Step 2, but is prohibited from querying the decryption oracle on the challenge ciphertext C^* .
5. $\mathcal{A}$ guesses a bit b' and wins the game if $b' = b$. $\mathcal{A}$'s advantage is defined as $\text{Adv}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$.

3.2 The Framework

Let $\Pi = (\text{Setup}, \text{VerifySetup}, \text{Encrypt}, \text{VerifyCT}, \text{PartialDec}, \text{VerifyPD}, \text{Decrypt})$ be a CCA2-secure threshold encryption cryptosystem. Figure 1 depicts the general $O(1)$ -verification PVSS framework based on Π .

Functionality	The Generic Framework
	$(\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \text{PVSS.Setup}(\lambda, t, n)$
	return Setup (λ, t, n)
	$\text{bool} \leftarrow \text{PVSS.SetupVrf}(\text{pp}, \{\text{pk}_i\})$
	return VerifySetup $(\text{pp}, \{\text{pk}_i\})$
	$(C, \pi_s) \leftarrow \text{PVSS.Share}(s, \text{pp})$
	return Encrypt (s, pp)
	$\text{bool} \leftarrow \text{PVSS.Verify}(C, \pi_s)$
	return VerifyCT (C, π_s)
	$D_i \leftarrow \text{PVSS.PreRecon}(C, \text{sk}_i)$
	return PartialDec (C, sk_i)
	$\text{bool} \leftarrow \text{PVSS.DecShareVrf}((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)$
	return VerifyPD $((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)$
	$S \leftarrow \text{PVSS.Recon}(C, \{D_i\}_{i \in Q})$
	return Decrypt $(C, \{D_i\}_{i \in Q})$

Fig. 1. Generic $O(1)$ -complexity PVSS framework based on CCATE

3.3 Security Analysis

As the PVSS algorithms directly invoke CCATE algorithms, shown in Figure 1. Most of the PVSS security properties can be implied by the implications of CCATE scheme.

- **Correctness** This is implied by **Correctness** and **Robustness** of CCATE scheme.
- **Public Verifiability**
 - **Verifiability of Setup** This is implied by **PV1** of CCATE scheme.
 - **Verifiability of Distribution** This is implied by **PV2** of CCATE scheme.
 - **Verifiability of Decryption** This is implied by **PV3** of CCATE scheme.
- **IND1-Secrecy** This is implied by the CCA2 game of CCATE scheme. More formally, denote $\mathcal{A}$ and $\mathcal{A}_{PVSS}$ as the adversaries in the games of CCA2-secure threshold encryption and PVSS schemes, respectively. If the PVSS framework is insecure, i.e., $\mathcal{A}_{PVSS}$ can gain non-negligible advantage in PVSS **IND1-security** game, then it can help $\mathcal{A}$ guess b in the ciphertext $(C^*, \pi_{s_b}) \leftarrow \text{Encrypt}(s_0, \text{pp})$ with high probability. This violates the definition of CCA2 security.

4 CCATE Based On Threshold ElGamal

4.1 Committee-based DKG

DKG is a fundamental cryptographic primitive that enables a group of n parties to jointly generate a shared secret key in a fully decentralized manner, without reliance on any trusted third party. In traditional DKG protocols, all n participants must actively engage in the process, exchanging messages with one another to establish the key pairs. However, this requirement leads to significant communication overhead and scalability challenges, especially when n is large.

This paper introduces the concept of Committee-based DKG, a novel approach that improves efficiency and trust distribution. Instead of requiring all n parties to directly communicate with each other in the DKG protocol, a committee of N members is designated to securely and efficiently perform the key generation on behalf of the entire group. This committee-based approach reduces communication overhead when n is large. Furthermore, it allows external parties, out of the group, to contribute to the DKG process, thereby enhancing overall trustworthiness in the generated key.

Definition 6 (Committee-based DKG). *An N -committee generates distributed keys for n parties in a decentralized manner, where the N committee members and the n parties are not necessarily overlapping. $t(\leq n)$ is a threshold value.*

1. *At the end of the protocol, each party obtains a pair of keys $(\text{sk}_i, \text{pk}_i)$. Also, a public key pk is jointly generated and made public. The corresponding secret key sk can be recovered by any honest subset I of t secret keys $\{\text{sk}_i\}_{i \in I}$.*
2. *The protocol is secure against a malicious adversary who can corrupt up to $t - 1$ of the n parties or at most $N - 1$ committee members, ensuring both the secrecy and correctness of the generated master secret key sk .*
3. *Any subset of t or more honest parties can collaborate to recover the master secret key sk .*

Implementation Figure 2 presents a committee-based DKG protocol, implemented based on Feldman VSS scheme [37] (refer to Appendix B). Besides, we assume that at least t parties and at least one committee member are honest. In practice, we set $t > n/2$ to ensure resilience against collusion attacks. A secure, authenticated communication channel exists between every committee member and each party. Additionally, a public bulletin board is available to record commitments and any complaints about share verification failures. We assume a synchronous network model, in which all messages are delivered within a known, bounded time. The n parties do not communicate directly with one another.

Functionality Committee-based DKG
$(\text{pp}, (\{\text{sk}_i, \text{pk}_i\})) \leftarrow \mathbf{DKG}(\lambda, t, n)$ – Each committee member i selects a random secret $k_i \in \mathbb{Z}_p$ and initiates a Feldman VSS sharing algorithm $\mathbf{VSS.Share}(k_i, n, t)$ to distribute shares $\{k_{i,j}\}_{j=1}^n$ to the n parties. Note that the commitment g^{k_i} is also published if the dealer is honest. – After all N committee members complete their sharing phases, all honest parties validate the correctness of the received shares via $\mathbf{VSS.Verify}$. Let $I \subseteq [1, N]$ denote the set of honest committee members, i.e., those against whom no valid complaints have been submitted. – The master public key pk is computed as the product of all individual commitments $\text{pk} = \prod_{i \in I} g^{k_i}$. – Each party P_j calculates its private key share as $\text{sk}_j = \sum_{i \in I} k_{i,j}$ and computes the corresponding public key share as $\text{pk}_j = g^{\text{sk}_j}$. – Optionally, the master secret key can be calculated with at least t partial secret keys $\{\text{sk}_j\}$ using Lagrange interpolation algorithm, i.e., $\text{sk} = \mathbf{VSS.Reconstruct}(\{(j, \text{sk}_j)\}_{j \in Q})$, where $Q \subseteq [1, n]$ and $Q \geq t$. – Denote $\text{pp} = (g, t, n, \text{pk})$ as public parameters.

Fig. 2. Committee-based DKG for n distributed parties

Security Properties The security requirements of the proposed committee-based DKG protocol are analyzed as below.

- **Correctness (C1):** The master secret key sk is uniquely defined by all honest committee members.

Proof : Each committee member employs Feldman VSS to distribute shares, enabling every shareholder to verify the validity of their received share using the provided public commitments. Furthermore, under the assumption of a synchronous network, a termination timeout can be established—allowing sufficient time for all committee members to complete the VSS distribution phase and for parties to submit complaints against any detected malicious behavior. As a result, only shares from honest committee members are accepted, and due to the homomorphic property of the Lagrange interpolation

algorithm. The sk is calculated as below:

$$\text{sk} = \text{Intpl}(\{(j, \text{sk}_j)\}_{j \in Q}) = \text{Intpl}(\{(j, \sum_{i \in I} k_{i,j})\}_{j \in Q}) = \sum_{i \in I} \text{Intpl}((j, k_{i,j})) = \sum_{i \in I} k_i.$$

□

- **Correctness (C2):** The master public key pk equals g^{sk} , where sk is the master secret key.

Proof : As proved in Correctness (C1), $\text{sk} = \sum_{i \in I} k_i$, then

$$\text{pk} = \prod_{i \in I} g^{k_i} = g^{\sum_{i \in I} k_i} = g^{\text{sk}}.$$

□

- **Correctness (C3):** The master private key sk is uniformly distributed in $\mathbb{Z}_p$ and the master public key is uniformly distributed in $\mathbb{G}$.

Proof : An honest committee member i selects their secret k_i uniformly at random, as prescribed by the DKG protocol. Consequently, the sum $\text{sk} = \sum_{i \in I} k_i$ is also uniformly distributed over $\mathbb{Z}_p$. Further, the public key $\text{pk} = g^{\text{sk}}$ is uniformly distributed over the group $\mathbb{G}$.

□

- **Secrecy:** The secret key sk must remain computationally hidden from any adversary that controls fewer than t parties, provided that at least one committee member is honest.

Proof : Correctness (C3) establishes that sk is uniformly distributed over its domain, provided there is at least one honest committee member, making it infeasible for adversaries to infer sk directly. Moreover, the honest member ensures that each party receives a share that is independently and uniformly random. The public key $\text{pk} = g^{\text{sk}}$ reveals no information about sk due to the hardness of the discrete logarithm problem. Furthermore, any coalition of at most $t - 1$ parties cannot reconstruct sk , as guaranteed by the information-theoretic security of Shamir's secret sharing scheme.

□

- **Robustness:** The corresponding secret key sk can be reconstructed by any subset of at least t sub keys $\{\text{sk}_j\}$.

Proof : This is implied by Correctness (C1), where sk is calculated by the Lagrange Interpolation algorithm.

□

- **Public Verifiability:** All commitments and complaints are published via a bulletin board, allowing any external party to verify the correctness of the DKG process without learning any private information.

Proof : VSS enables the n parties to verify their respective shares against the public commitments. A committee member is considered malicious and its shares are discarded if a valid complaint is raised. To prevent malicious complaints, we require that all shares distributed to parties are properly signed by the committee member. Consequently, any complaint can be publicly verified by checking the authenticity of the member's signature on the

disputed share. The combination of commitments in the distribution phase and signature-verified complaints in the complaint phase ensures that the entire DKG protocol is publicly verifiable. $\square$

Dynamicity of Parties To support dynamicity of parties in the DKG protocol, any t -out-of- n existing parties can reshare their sk_j to shift a new (t', n') -DKG group, maintaining (sk, pk) unchanged. This idea of dynamicity is rooted in proactive secret sharing [40] and resharing mechanism [41]. Let J denote the set of t existing parties, and let L be the new group with $|L| = n'$. The implementation of dynamicity can be described as follows using Feldman VSS scheme.

- Each party $P_j \in J$ runs the Feldman VSS algorithm $\text{VSS.Share}(\text{sk}_j, t', n')$, generating shares $\{\text{sk}_{j,l}\}_{l \in [1, n']}$ for each new party $P_l \in L$.
- Note that the VSS commitment g^{sk_j} should equal $\prod_{i \in I} g^{k_i}$, where g^{k_i} are available in the committee-based DKG, ensuring that P_j correctly reshapes the secret sk_j . Furthermore, each party in L invokes the VSS.Verify algorithm to verify the correctness of the received share $\text{sk}_{j,l}$.
- Suppose that t parties, forming a subset $Q \subseteq J$, successfully complete the sharing process, and no valid complaints are raised.
- Then, each party P_l in L computes his fresh secret key as $\text{sk}_l = \text{Intpl}(\{j, \text{sk}_{j,l}\})$. The dynamicity is correct as: $\text{sk}' = \text{Intpl}(\{l, \text{sk}_l\}) = \text{Intpl}(\{l, \text{Intpl}(\{j, \text{sk}_{j,l}\})\}) = \text{Intpl}(\{j, \text{Intpl}(\{l, \text{sk}_{j,l}\})\}) = \text{Intpl}(\{j, \text{sk}_j\}) = \text{sk}$.

4.2 CCATE Instantiation I

Figure 3 presents the implementation of a CCATE scheme based on committee-based DKG and Threshold ElGamal. Note that the implementation is pairing-free. To better present and comply with the PVSS definition, we incorporate $s \in \mathbb{Z}_p$ as the ElGamal plaintext. Then, the **Encrypt** algorithm produces ElGamal ciphertext C for s . It also generates an NIZK proof π_s using Sigma protocol and Fiat-Shamir heuristic, proving that the C is well formed and decryptable. Further, we leverage *DLEQ* in **PartialDec** algorithm to enable shareholders to prove correctness of decrypted partial shares.

4.3 Security Analysis

We prove that the construction depicted in Figure 3 achieves **Correctness**, **Publicly verifiability**, **CCA2 security**. Further, by the framework established in Section 3, the construction implies a secure PVSS construction.

Lemma 1 (NIZK Proof Completeness I). *If an encryptor is honest, then $\Pr[(C, \pi_s) \leftarrow \text{Encrypt}(s, \text{pp}) : \text{VerifyCT}(C, \pi_s) = 1] = 1$.*

Functionality CCA2-Secure Threshold ElGamal

$(\text{pp}, (\{\text{sk}_i, \text{pk}_i\})) \leftarrow \mathbf{Setup}(\lambda, t, n)$

return $\mathbf{DKG}(\lambda, t, n)$

$(\text{pp}, (\{\text{sk}_i, \text{pk}_i\})) \leftarrow \mathbf{VerifySetup}(\text{pp}, \{\text{pk}_i\})$

This is implied by $\mathbf{DKG}(\lambda, t, n)$. More specifically, it relies on verification of VSS shares and complaints from honest parties.

$(C, \pi_s) \leftarrow \mathbf{Encrypt}(s, \text{pp})$

$w, w' \xleftarrow{\$} \mathbb{Z}_p, s' \xleftarrow{\$} \mathbb{Z}_p$

$C = (g^s \cdot \text{pk}^w, g^w), C' = (g^{s'} \cdot \text{pk}^{w'}, g^{w'})$

$c = H(C, C'), \hat{w} = w' - wc, \hat{s} = s' - cs$

$\pi_s = (C', c, (\hat{w}, \hat{s}))$

$\text{bool} \leftarrow \mathbf{VerifyCT}(C, \pi_s)$

parse π_s as $(C', c, (\hat{w}, \hat{s}))$

$C'[0] \stackrel{?}{=} C[0]^c \cdot g^{\hat{s}} \cdot \text{pk}^{\hat{w}}$

$C'[1] \stackrel{?}{=} C[1]^c \cdot g^{\hat{w}}$

$D_i \leftarrow \mathbf{PartialDec}(C, \text{sk}_i)$

$D_i = C[1]^{\text{sk}_i}$

$\pi_{\text{sk}_i} = \mathbf{DLEQ}(g, \text{pk}_i, C[1], D_i, \text{sk}_i)$

$\text{bool} \leftarrow \mathbf{VerifyPD}((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)$

$\text{true} \stackrel{?}{=} \mathbf{DLEQVrf}(g, \text{pk}_i, C[1], D_i, \pi_{\text{sk}_i})$

$S \leftarrow \mathbf{Decrypt}(C, \{D_i\}_{i \in Q})$

$S \leftarrow \frac{C[0]}{\prod_{i \in Q} D_i^{L_i(0)}}$, where $L_i(x)$ is the Lagrange basis polynomial.

Fig. 3. CCATE based on committed-based DKG setup and Threshold ElGamal

Proof : We check the two judgment equations in the **VerifyCT** algorithm as follows:

$$C[0]^c \cdot g^{\hat{s}} \cdot \text{pk}^{\hat{w}} = (g^s \text{pk}^w)^c \cdot g^{s' - cs} \cdot \text{pk}^{w' - wc} = g^{s'} \text{pk}^{w'} = C'[0],$$

$$C[1]^c \cdot g^{\hat{w}} = (g^w)^c \cdot g^{w' - wc} = g^{w'} = C'[1].$$

Further, completeness indicates that the ciphertext is well formed. $\square$

Lemma 2 (NIZK Proof Soundness I). *Given two accepting transcripts $(C, \pi_{s1} = (C', c_1, (\hat{w}_1, \hat{s}_2)))$ and $(C, \pi_{s2} = (C', c_2, (\hat{w}_2, \hat{s}_2)))$, an efficient extractor can output the witness s and w .*

Proof : Since the two transcripts are accepting, we have

$$\hat{w}_i = w' - wc_i, \quad \hat{s}_i = s' - c_i s,$$

where $i = 1, 2$. Then, by subtracting the equations from each other, we obtain

$$\hat{w}_1 - \hat{w}_2 = (c_1 - c_2)w, \quad \hat{s}_1 - \hat{s}_2 = (c_1 - c_2)s.$$

Since $c_1 \neq c_2$, we can invert $(c_1 - c_2)$ in the underlying field, yielding:

$$w = (\hat{w}_1 - \hat{w}_2)(c_1 - c_2)^{-1}, \quad s = (\hat{s}_1 - \hat{s}_2)(c_1 - c_2)^{-1}.$$

This extraction is computable and efficient. The resulting (w, s) satisfies the witness relation by construction, completing the proof. $\square$

Theorem 1 (Correctness). *The CCATE described in Figure 3 satisfies correctness, defined by Definition 2.*

Proof : By Lemma 1 and Lemma 2, it can be inferred that only honest parties can produce a well-formed/valid ciphertext (C, π_s) . Then, we prove that a well-formed ciphertext can be decrypted to S correctly, such that $(s, S) \in \mathcal{PK}$. Suppose there are $|Q| \geq t$ partial decrypted shares correctly generated by the **PartialDec** algorithm, i.e., $C[1]^{\text{sk}_i}_{i \in Q}$. Then the **Decrypt** algorithm is correct as:

$$\frac{C[0]}{\prod_{i \in Q} D_i^{L_i(0)}} = \frac{g^s \text{pk}^w}{\prod_{i \in Q} (g^w)^{\text{sk}_i L_i(0)}} = \frac{g^s \text{pk}^w}{(g^w)^{\sum_{i \in Q} \text{sk}_i L_i(0)}} = \frac{g^s \text{pk}^w}{(g^w)^{\text{sk}}} = g^s = S.$$

Obviously, $\Pr[(s, S) \in \mathcal{PK}] = 1$. $\square$

Theorem 2 (Robustness). *The CCATE described in Figure 3 satisfies robustness, defined by Definition 3.*

Proof : To prove robustness, we argue that attempts to manipulate decryption shares fail to affect the correct reconstruction of the secret S . We leverage the **VerifyPD** algorithm to check whether a partial decryption share D_i is valid or not using π_{sk_i} . The algorithm reveals no information about sk_i , as DLEQ relies on the discrete logarithm problem and hash oracle. Further, for any two valid t -sized sets of decryption shares Q and Q' , the reconstructed secret S is identical due to the uniqueness of Lagrange interpolation algorithm. $\square$

Theorem 3 (Public Verifiability). *The CCATE described in Figure 3 satisfies public verifiability, defined by Definition 4.*

Proof : Public verifiability in CCATE is proven for each phase of the protocol.

- Property **PV1** follows from the Public Verifiability of the committee-based DKG, as established in Section 4.1.
- Property **PV2** is implied by the completeness and soundness of the NIZK proof system, as shown in Lemma 1 and Lemma 2.
- Property **PV3** is addressed in the proof of Theorem 2, where it is shown that no adversary can forge a valid decryption share.

$\square$

Lemma 3 (NIZK Proof Zero-Knowledge I). *Given input (C, c) , a simulator can output a transcript $(\tilde{C}', c, (\tilde{w}, \tilde{s}))$ identical to real-world output between an honest dealer and an honest shareholder.*

Proof : We construct a simulator that, given $C = (C[0], C[1]) = (g^s \cdot \text{pk}^w, g^w)$ and a challenge $c \in \mathbb{Z}_p$, outputs a valid transcript $(\tilde{C}', c, (\tilde{w}, \tilde{s}))$ without knowing s or w .

1. Sample random values: $\tilde{w} \xleftarrow{\$} \mathbb{Z}_p$, $\tilde{s} \xleftarrow{\$} \mathbb{Z}_p$.
2. Compute the fake commitment $\tilde{C}'$ as: $\tilde{C}' = (g^{\tilde{s}} \cdot \text{pk}^{\tilde{w}} \cdot C[0]^c, g^{\tilde{w}} \cdot C[1]^c)$.
This ensures that the verification equation will pass with the response $(\tilde{w}, \tilde{s})$ and challenge c .
3. Output the simulated transcript: $(\tilde{C}', c, (\tilde{w}, \tilde{s}))$.

In the real protocol, w, w', s' are sampled uniformly at random in $\mathbb{Z}_p$. $c = H(C, C')$ is determined by the random oracle. Consequently, $\hat{w} = w' - wc$, $\hat{s} = s' - sc$ are uniform (due to one-time pad-like masking), and C' is a uniformly random group element in the image of the commitment. In the simulation, the simulator samples $\tilde{w}$ and $\tilde{s}$ uniformly from $\mathbb{Z}_p$, resulting in distributions identical to $(\hat{w}, \hat{s})$. $\tilde{C}'$ is computed as a uniformly random-looking group element (since $g^{\tilde{s}}, g^{\tilde{w}}$ are random). Thus, the distribution of $(\tilde{C}', c, (\tilde{w}, \tilde{s}))$ is identical to that of a real transcript, because all components are sampled from the same distributions and satisfy the same algebraic relations. $\square$

Theorem 4. *The CCATE described in Figure 3 is **CCA2** secure against the game in Definition 5 under the Decisional Diffie-Hellman (DDH) assumption in the random oracle model.*

Proof : We use a sequence of games starting with the real attack game and ending with a game where the adversary $\mathcal{A}$ has no advantage against the DDH assumption. Denote E_j as the event that $\mathcal{A}$ wins in each **Game** _{j} .

- **Game**₀: This is the real attack game, where an adversary $\mathcal{A}$ can simulate an indistinguishability game (defined by Definition 5) as follows:
 1. *Setup*: The challenger runs the **Setup** algorithm and denotes $\text{pk} = g^\alpha$. For $t \leq i \leq n$, $\mathcal{A}$ selects uniformly random values $u_i \leftarrow \mathbb{Z}_p$ and sends the values $\text{pk}_i = g^{u_i}$ to $\mathcal{A}$.
 2. *Query*: (Without loss of generality, we assume that $\mathcal{A}$ corrupts the first $t - 1$ parties.)
 - For $1 \leq i \leq t - 1$, the challenger sends sk_i to $\mathcal{A}$.
 - For polynomial times, $\mathcal{A}$ inquires a valid ciphertext C and the challenger responds $D_i \leftarrow \text{PartialDec}(C, \text{sk}_i)$ for $i \in [1, n]$.
 3. Choosing s_0 and s_1 randomly, the challenger calculates a challenge ciphertext $C^* = (g^{s_0} \cdot \text{pk}^w, g^w)$ and $\pi_{s_0}^* = (C'^*, c^*, \{\hat{w}^*, \hat{s}^*\})$ and sends (s_b, C^*) to $\mathcal{A}$, where $b \xleftarrow{\$} [0, 1]$.
 4. $\mathcal{A}$ makes additional queries, but is prohibited to make decryption queries on C^* .
 5. $\mathcal{A}$ makes a guess b' . If $b' = 0$, $\mathcal{A}$ guesses that C^* is generated from s_b , otherwise, it guesses C^* is not generated from s_b .

- **Game₁**: The NIZK proof $\pi_{s_0}^*$ reveals no information about s_b or b by Lemma 3. This game modifies **Game₀** by changing $\pi_{s_0}^*$ to randomly chosen elements. This modification cannot affect $\mathcal{A}$'s behavior in guessing b , i.e., $|\Pr[E_1] - \Pr[E_0]| \leq \text{negl}(\lambda)$.
- **Game₂**: In the Query phases, Lemma 3 also ensures that any ciphertext C , generated from a fresh plaintext, is uniformly distributed in $\mathbb{G}$. Further, the partial decryption share of C , i.e., $D_i = C[1]^{\text{sk}_i}$, reveals no information about sk_i . Therefore, the Query phases reveal no information about the challenge plaintext s_b or b . This game assumes the queried results are randomly chosen elements. This modification also does not affect $\mathcal{A}$'s behavior, i.e., $|\Pr[E_2] - \Pr[E_1]| \leq \text{negl}(\lambda)$.
- **Game₃**: $\mathcal{A}$ can additionally calculate **PartialDec**(C^*, sk_i) to obtain $D_i^* = g^{w \cdot \text{sk}_i}$ where $i \in [1, t-1]$. The information-theoretic secrecy of Shamir secret sharing scheme shows that $\{\text{sk}_i\}_{i \in [1, t-1]}$ reveals no information about sk . Consequently, $\{D_i\}_{i \in [1, t-1]}$ reveals nothing information about $g^{w \cdot \text{sk}}$ or pk^w , which is essential to decrypt C^* . This game modifies $\{D_i\}_{i \in [1, t-1]}$ to random elements from $\mathbb{G}^{t-1}$. This modification implies $|\Pr[E_3] - \Pr[E_2]| \leq \text{negl}(\lambda)$.

The following games prove that ElGamal ciphertext $C^* = (g^{s_0} \cdot \text{pk}^w, g^w)$ is secure based on DDH assumption. Denote $\text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda)$ as the advantage of the best PPT adversary against the DDH problem.

- **Game₄**: This game modifies pk^w to $K \xleftarrow{\$} \mathbb{G}$ for the ciphertext C^* . This modification does not impact randomness of the ciphertext as w is randomly chosen from $\mathbb{Z}_p$. This modification is justified by the DDH assumption. Specifically, the tuples $(g, \text{pk} = g^{\text{sk}}, g^w, \text{pk}^w)$ and $(g, \text{pk} = g^{\text{sk}}, g^w, K)$, are computationally indistinguishable. The DDH assumption implies that no efficient adversary can distinguish them with non-negligible advantage. Therefore, $|\Pr[E_5] - \Pr[E_4]| \leq \text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda)$.
- **Game₅**: This game modifies $C^*[0]$ to be another random element in $\mathbb{G}$. Consequently, $C^* = (K', g^w)$, where $K' \xleftarrow{\$} \mathbb{G}, w \xleftarrow{\$} \mathbb{Z}_p$. Hence, $|\Pr[E_5] - \Pr[E_4]| \leq \text{negl}(\lambda)$.
- **Game₆**: This game further modifies $C^*[0]$ to be an element generated from s_1 (i.e., the guess value $b = 1$). Hence, $C^* = (g^{s_1} K, g^w)$, where $K \xleftarrow{\$} \mathbb{G}, w \xleftarrow{\$} \mathbb{Z}_p$. Thus, $|\Pr[E_6] - \Pr[E_5]| \leq \text{negl}(\lambda)$.
- **Game₇**: This game sets $K = \text{pk}^w$, resulting in the ciphertext $C^* = (g^{s_1} \text{pk}^w, g^w)$. This game also assumes DDH assumption, where the two indistinguish tuples are $(g, \text{pk} = g^{\text{sk}}, g^w, K)$ and $(g, \text{pk} = g^{\text{sk}}, g^w, \text{pk}^w)$. Hence, $|\Pr[E_7] - \Pr[E_6]| \leq \text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda)$.

Therefore, $|\Pr[E_0] - \Pr[E_7]| \leq 2 \cdot \text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda)$. In other words, even given the $t-1$ corrupted secret keys $\{\text{sk}_i\}_{i \in [1, t-1]}$, the challenge ciphertext C^* , and the NIZK proof $\pi_{s_0}^*$, the bit b and the value s_b remain computationally hidden. More precisely, any PPT adversary has at most negligible advantage in guessing b , so $\text{Adv}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}| \leq 2 \cdot \text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda)$. $\square$

5 CCATE Based On Silent Setup

We present the second CCATE Instantiation by leveraging the CPA-Secure silent setup encryption (STE) scheme [28] and NIZK proofs.

5.1 Silent Setup

Power-Of-Tau with Public Verifiability The “Power-of-Tau” refers to a common reference string (CRS) used in pairing-based zero-knowledge proofs. Particularly, the goal is to generate a CRS containing powers of a secret value τ , which is unknown to anyone and $\text{CRS} = (\{g_1^{\tau^i}, g_2^{\tau^i}\}_{i \in [1, n]})$, where g_1 is a random generator in $\mathbb{G}_1$, g_2 is a random generator in $\mathbb{G}_2$ and n is the maximum degree of polynomials used in the proof system.

Algorithm 1 presents a publicly verifiable construction to generate CRS as long as at least one party is honest. Specifically, each party can invoke *PowerOfTau* to update the CRS and anyone can check whether an updated CRS is valid or not via *VerifyCRS*.

Algorithm 1 Power of Tau to calculate CRS and its verification

1: func <i>PowerOfTau</i> (CRS, β)	1: func <i>VerifyCRS</i> (CRS)
2: Parse CRS as $\{g_1^{\tau^i}, g_2^{\tau^i}\}_{i \in [1, n]}$	2: Parse CRS as $\{g_1^{\tau^i}, g_2^{\tau^i}\}_{i \in [1, n]}$
3: while $i \in [1, n]$ do	3: while $i \in [1, n]$ do
4: $g_1^{\tau^i} \leftarrow (g_1^{\alpha^i})^{\beta^i}$	4: if $e(g_1, g_2^{\tau^i}) \neq e(g_1^{\tau^i}, g_2)$ then
5: $g_2^{\tau^i} \leftarrow (g_2^{\alpha^i})^{\beta^i}$	5: return false
6: end while	6: end if
7: Destroy β	7: end while
8: return $\{g_1^{\tau^i}, g_2^{\tau^i}\}_{i \in [1, n]}$	8: return true
9:	9:

Silent Setup for DKG pairs To make the writing consistency, we describe the setup in DKG-style which primarily outputs threshold-enabled key pairs. Figure 4 shows the publicly verifiable DKG construction, which plays the role of silent setup. Note that this DKG does not require Shamir SS or VSS scheme. The setup phase **Setup** employs the Power of Tau protocol to produce a common reference string CRS, relying on the assurance that at least one participant in the ceremony is honest. Additionally, each party P_i generates a key pair $(\text{sk}_i, \text{pk}_i)$ locally, where the public key pk_i enables **VerifySetup** to verify the integrity and correctness of P_i ’s actions. $L(x)$ is the Lagrange basis polynomial and $Z(x)$ is the vanishing polynomial defined in Appendix C

Dynamicity of Parties The silent setup inherently supports dynamic changes of parties, as each party can independently generate its own key pair. A party

Functionality CCATE with Silent Setup

$(\text{pp}, (\{\text{sk}_i, \text{pk}_i\})) \leftarrow \text{Setup}(\lambda, t, n)$

- Sample $g_1 \xleftarrow{\$} \mathbb{G}_1, g_2 \xleftarrow{\$} \mathbb{G}_2$, compute $g_t = e(g_1, g_2)$ and sample $g'_t \xleftarrow{\$} \mathbb{G}_T$.
- Compute $(n+1)$ -th root of unity ω such that $\omega^{n+1} = 1$. Obtain $\text{CRS} = (\{g_1^{\tau^i}, g_2^{\tau^i}\}_{i \in [1, n+1]})$ via *PowerOfTau* and *VerifyCRS* in Algorithm 1.
- Each party P_i randomly samples $\text{sk}_i \xleftarrow{\$} \mathbb{Z}_p$, computes and publishes $\text{pk}_{i,1} = g_1^{\text{sk}_i}$ and $\text{pk}_{i,2} = (h_{i,1}, h_{i,2}, h_{i,3}, h_{i,4}, \{h_{i,5,j}\}_{j \in [0, n], j \neq i})$, where $h_{i,1} = g_1^{\text{sk}_i L_i(\tau)}$, $h_{i,2} = g_1^{\text{sk}_i (L_i(\tau) - L_i(0))}$, $h_{i,3} = g_1^{\text{sk}_i \frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)}}$, $h_{i,4} = g_1^{\text{sk}_i \frac{(L_i(\tau) - L_i(0))}{\tau}}$, $\{h_{i,5,j} = g_1^{\text{sk}_i \frac{L_i(\tau) L_j(\tau)}{Z(\tau)}}\}$.
- Outputs $\text{pp} = (\text{CRS}, \{\omega^i\}, g_1, g_2, g_t, g'_t)$ and $\{\text{sk}_i, \text{pk}_i = (\text{pk}_{i,1}, \text{pk}_{i,2})\}$.

$\text{bool} \leftarrow \text{VerifySetup}(\text{pp}, \{\text{pk}_i\})$

- For each $i \in [1, n]$, an honest-but-curious aggregator verifies each $\text{pk}_{i,2}$:

$$e(h_{i,1}, g_2) \stackrel{?}{=} e(\text{pk}_{i,1}, g_2^{L_i(\tau)}), e(h_{i,2}, g_2) \stackrel{?}{=} e(\text{pk}_{i,1}, g_2^{(L_i(\tau) - L_i(0))}), e(h_{i,3}, g_2) \stackrel{?}{=} e(\text{pk}_{i,1}, g_2^{\frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)}}), e(h_{i,4}, g_2) \stackrel{?}{=} e(\text{pk}_{i,1}, g_2^{\frac{(L_i(\tau) - L_i(0))}{\tau}}), e(h_{i,5,j}, g_2) \stackrel{?}{=} e(\text{pk}_{i,1}, g_2^{\frac{L_i(\tau) L_j(\tau)}{Z(\tau)}}).$$
- Let I be the index set with valid public keys. Set $\text{sk}_0 = 1_{\mathbb{Z}_p}$ for the dummy party with index 0.
- The aggregator computes $X = \prod_{i \in I} h_{i,1}, Z = g_2^{Z(\tau)}, A_i = \prod_{j=0, j \neq i}^n h_{j,5,i} (i \in I \cup \{0\})$ and sets $\text{ek} = (X, Z), \text{ak} = (I, \{\text{pk}_i\}_{i \in I \cup \{0\}}, \{h_{i,2}\}_{i \in I \cup \{0\}}, \{h_{i,3}\}_{i \in I \cup \{0\}}, \{h_{i,4}\}_{i \in I \cup \{0\}}, \{A_i\}_{i \in I \cup \{0\}})$.
- Finally, the algorithm updates $\text{pp} \leftarrow \text{pp} \cup \{(\text{ek}, \text{ak})\}$.

Fig. 4. Silent setup for n distributed parties

becomes an active member of the system once its public key is validated and incorporated into the aggregated keys ek and ak .

5.2 CCATE Instantiation II

Leveraging the silent setup outputs in Section 5.1, Figure 5 illustrates the following algorithms of CCATE scheme. Our primary contribution lies in the construction of NIZK proofs integrated into the STE scheme [28] (see Appendix F for details). These NIZK proofs π_s are derived using the Sigma protocol combined with the Fiat-Shamir heuristic. Specifically, $\pi_s = (C', c, (\hat{\gamma}, \hat{\gamma}, \hat{\mathbf{v}}, \hat{s}))$, where C' is the commitment, c is the challenge, and $(\hat{\gamma}, \hat{\gamma}, \hat{\mathbf{v}}, \hat{s})$ are the response values.

5.3 Security Analysis

We prove that the construction depicted in Figure 5 achieves **Correctness**, **Publicly verifiability**, **CCA2 security**.

Functionality CCATE from Silent Setup

$(C, \pi_s) \leftarrow \mathbf{Encrypt}(s, \text{pp})$

$\gamma, \gamma' \xleftarrow{\$} \mathbb{Z}_p$, set $Z_0 = g_2^{\tau-1}$, parse $\text{ek} = (X, Z)$ and set $\mathbf{A} =$

$$\begin{matrix} X & g_2 & Z & g_2^\tau & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & g_2^\tau & g_2 & 0 & 0 & 0 \\ 0 & g_2^\gamma & 0 & 0 & 0 & g_1 & 0 & 0 \\ g_1^{\tau^t} & 0 & 0 & 0 & 0 & 0 & g_2 & 0 \\ g_1 & 0 & 0 & 0 & 0 & 0 & 0 & Z_0 \end{matrix}$$

set $\mathbf{b} = (1_{\mathbb{G}_T}, 1_{\mathbb{G}_T}, 1_{\mathbb{G}_T}, 1_{\mathbb{G}_T}, g_t)^\top$, $\mathbf{v} = (s_1, \dots, s_5)^\top \xleftarrow{\$} \mathbb{Z}_p^5$, $\mathbf{v}' = (s'_1, \dots, s'_5)^\top \xleftarrow{\$} \mathbb{Z}_p^5$

$C = (g_2^\gamma, \mathbf{A}^\mathbf{v}, \mathbf{b}^\mathbf{v} \cdot g_t^s)$, where $\mathbf{A}^\mathbf{v}$ and $\mathbf{b}^\mathbf{v}$ denote scalar exponentiation in matrix.

$C' = (g_2^{\gamma'}, \mathbf{A}^{\mathbf{v}'}, \mathbf{b}^{\mathbf{v}'} \cdot g_t^{s'})$

$c = H(C, C')$, $\hat{\gamma} = \gamma' - c\gamma$, $\hat{\mathbf{v}} = \mathbf{v}' - c\mathbf{v}$, $\hat{s} = s' - cs$

$\pi_s = (C', c, \hat{\gamma}, \hat{\mathbf{v}}, \hat{s})$

$\text{bool} \leftarrow \mathbf{VerifyCT}(C, \pi_s)$

Parse $\pi_s = (C', c, \hat{\gamma}, \hat{\mathbf{v}}, \hat{s})$

Verify $c \stackrel{?}{=} H(C, C')$, $C'[0] \stackrel{?}{=} C[0]^c g_2^{\hat{\gamma}}$, $C'[1] \stackrel{?}{=} C[1]^c \circ \mathbf{A}^{\hat{\mathbf{v}}}$, $C'[2] \stackrel{?}{=} C[2]^c \mathbf{b}^{\hat{\mathbf{v}}} g_t^{\hat{s}}$, where $\circ$ denotes hadamard product.

$D_i \leftarrow \mathbf{PartialDec}(C, \text{sk}_i)$

$D_i = C[0]^{\text{sk}_i} = g_2^{\gamma^{\text{sk}_i}}$

$\pi_{\text{sk}_i} = \text{DLEQ}(g, \text{pk}_i, C[0], D_i, \text{sk}_i)$

$\text{bool} \leftarrow \mathbf{VerifyPD}((D_i, \pi_{\text{sk}_i}), \text{pk}_i, C)$

$\text{true} \stackrel{?}{=} \text{DLEQVrf}(g, \text{pk}_i, C[0], D_i, \pi_{\text{sk}_i})$

$S \leftarrow \mathbf{Decrypt}(C, \{D_i\}_{i \in P})$

Given ak and enough $\{D_i\}$, this algorithm computes the vector $\mathbf{w}^a$ such that $e(\mathbf{A}, \mathbf{w}) = \mathbf{b}$ and outputs $S = \frac{C[2]}{e(C[1], \mathbf{w})}$.

^a Refer to Appendix F for the calculation of $\mathbf{w}$

Fig. 5. CCATE algorithms given the silent setup outputs $(\text{pp}, (\{\text{sk}_i, \text{pk}_i\}))$ in Figure 4

Lemma 4 (NIZK Proof Completeness II). *If an encryptor is honest, then $\Pr[(C, \pi_s) \leftarrow \mathbf{Encrypt}(s, \text{pp}) : \mathbf{VerifyCT}(C, \pi_s) = 1] = 1$.*

Proof : Obviously, the check of the challenge value c is correct. Then, we check the equations in the **VerifyCT** algorithm as follows:

$$C[0]^c \cdot g_2^{\hat{\gamma}} = g_2^{\gamma^c} g_2^{\gamma' - \gamma^c} = g_2^{\gamma'} = C'[0]$$

$$C[1]^c \circ \mathbf{A}^{\hat{\mathbf{v}}} = \mathbf{A}^{\mathbf{v}^c} \circ \mathbf{A}^{\mathbf{v}' - \mathbf{v}^c} = \mathbf{A}^{\mathbf{v}'} = C'[1]$$

$$C[2]^c \mathbf{b}^{\hat{\mathbf{v}}} g_t^{\hat{s}} = \mathbf{b}^{\mathbf{v}^c} g_t^{s^c} \mathbf{b}^{\mathbf{v}' - \mathbf{v}^c} g_t^{s' - s^c} = \mathbf{b}^{\mathbf{v}'} g_t^{s'} = C'[2]$$

Further, completeness indicates that the ciphertext is well-formed. $\square$

Lemma 5 (NIZK Proof Soundness II). *Given two accepting transcripts $((C, \pi_{s1} = (C', c_1, \hat{\gamma}_1, \hat{\mathbf{v}}_1, \hat{s}_1)))$ and $((C, \pi_{s2} = (C', c_2, \hat{\gamma}_2, \hat{\mathbf{v}}_2, \hat{s}_2)))$, an efficient extractor can output the witness $\gamma, \mathbf{v}$ and s .*

Proof : Since the two transcripts are accepting, then we have

$$\hat{\gamma}_i = \gamma' - c_i \gamma, \quad \hat{\mathbf{v}}_i = \mathbf{v}' - c_i \mathbf{v}, \quad \hat{s}_i = s' - c_i s.$$

where $i = 1, 2$. Then, by subtracting the respective equations to have:

$$\hat{\gamma}_1 - \hat{\gamma}_2 = (c_2 - c_1) \gamma, \quad \hat{\mathbf{v}}_1 - \hat{\mathbf{v}}_2 = (c_2 - c_1) \mathbf{v}, \quad \hat{s}_1 - \hat{s}_2 = (c_2 - c_1) s.$$

Since $c_1 \neq c_2$, we can invert $(c_2 - c_1)$ in the underlying field, yielding:

$$\gamma = (\hat{\gamma}_1 - \hat{\gamma}_2)(c_2 - c_1)^{-1}, \quad \mathbf{v} = (\hat{\mathbf{v}}_1 - \hat{\mathbf{v}}_2)(c_2 - c_1)^{-1}, \quad s = (\hat{s}_1 - \hat{s}_2)(c_2 - c_1)^{-1}.$$

This extraction is computable and efficient. The resulting $(\gamma, \mathbf{v}, s)$ satisfies the witness relation by construction, completing the proof. $\square$

Theorem 5 (Correctness). *The CCATE described in Figure 5 satisfies correctness, defined by Definition 2.*

Proof : Lemma 4 and Lemma 5 ensure only well-formed/valid ciphertext (C, π_s) can pass the **VerifyCT** algorithm. Then, we prove that a well-formed ciphertext can be decrypted to S correctly, such that $(s, S) \in \mathcal{PK}$. If the public key $\mathbf{pk}_i$ is correctly verified by the algorithm **VerifySetup**, then $\mathbf{pk}_{i,2} = (h_{i,1}, h_{i,2}, h_{i,3}, h_{i,4}, \{h_{i,5,j}\})$ is well-formed by applying the pairing checks. If the partial decrypted share D_i is correctly verified by the **VerifyPD** algorithm, we get that $D_i = g_2^{\gamma \mathbf{sk}_i}$ is well-formed. The calculation of $\mathbf{w}$ is described in Appendix F, and the correctness of $\mathbf{w}$ is ensured by the correctness of the univariate sumcheck protocol (i.e., Lemma 7). Then the **Decrypt** algorithm is correct as:

$$\frac{C[2]}{e(C[1], \mathbf{w})} = \frac{\mathbf{b}^{\mathbf{v}} \cdot g_t'^s}{e(\mathbf{A}^{\mathbf{v}}, \mathbf{w})} = \frac{\mathbf{b}^{\mathbf{v}} \cdot g_t'^s}{e(\mathbf{A}, \mathbf{w})^{\mathbf{v}}} = \frac{\mathbf{b}^{\mathbf{v}} \cdot g_t'^s}{\mathbf{b}^{\mathbf{v}}} = g_t'^s = S.$$

Obviously, $\Pr[(s, S) \in \mathcal{PK}] = 1$. $\square$

Theorem 6 (Robustness). *The CCATE described in Figure 5 satisfies robustness, defined by Definition 3.*

Proof : We argue that attempts to manipulate decryption shares fail to affect the correct reconstruction of the secret S . We leverage the **VerifyPD** algorithm to check whether a partial decryption share D_i is valid or not using $\pi_{\mathbf{sk}_i}$. If $(D_i, \pi_{\mathbf{sk}_i})$ is correctly verified, we get that $D_i = g_2^{\gamma \mathbf{sk}_i}$ is well-formed, except with negligible probability (i.e., DLEQ soundness error). If the robustness is not satisfied, i.e., $\Pr[(\mathbf{Decrypt}(C, \{\mathbf{PartialDec}(C, \mathbf{sk}_i)\}_{i \in Q}) \neq \mathbf{Decrypt}(C, \{\mathbf{PartialDec}(C, \mathbf{sk}_i)\}_{i \in Q'})]$ is not negligible, then there exists:

$$S_Q = \mathbf{Decrypt}(C, \mathbf{PartialDec}(C, \{\mathbf{sk}_i\}_{i \in Q})) \wedge S_{Q'} = \mathbf{Decrypt}(C, \mathbf{PartialDec}(C, \{\mathbf{sk}_i\}_{i \in Q'})) \wedge S_Q \neq S_{Q'}.$$

From the correctness, we have $\Pr[(s, \mathbf{Decrypt}(C, \{\mathbf{PartialDec}(C, \mathbf{sk}_i)\}_{i \in Q})) \in \mathcal{PK}] = 1$ and $\Pr[(s, \mathbf{Decrypt}(C, \{\mathbf{PartialDec}(C, \mathbf{sk}_i)\}_{i \in Q'})) \in \mathcal{PK}] = 1$. Then, we have $(s, S_Q) \in \mathcal{PK}$ and $(s, S_{Q'}) \in \mathcal{PK}$, where $S_Q \neq S_{Q'}$, which violates the definition of $\mathcal{PK}$. Therefore:

$$\Pr[\mathbf{Decrypt}(C, \mathbf{PartialDec}(C, \{\mathbf{sk}_i\}_{i \in Q})) \neq \mathbf{Decrypt}(C, \mathbf{PartialDec}(C, \{\mathbf{sk}_i\}_{i \in Q'}))] < \text{negl}(\lambda).$$

□

Theorem 7 (Public Verifiability). *The CCATE described in Figure 5 satisfies public verifiability, defined by Definition 4.*

Proof : Public verifiability in CCATE is proven for each phase of the protocol.

- Property **PV1** is implied by the public verifiability of the Power-Of-Tau ceremony and the public keys from silent setup, which is established in Section 5.1.
- Property **PV2** is implied by the completeness and soundness of the NIZK proof system, as shown in Lemma 4 and Lemma 5.
- Property **PV3** is achieved by the **VerifyPD** algorithm in Figure 5. We leverage the DLEQ NIZK protocol to verify the partial decryption share publicly.

□

Lemma 6 (NIZK Proof Zero-Knowledge II). *Given input (C, c) , a simulator can output a transcript $((\tilde{C}', c, \tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s}))$ identical to real-world output between an honest dealer and an honest shareholder.*

Proof : We construct a simulator that, given $C = (C[0], C[1], C[2]) = (g_2^\gamma, \mathbf{A}^\mathbf{v}, \mathbf{b}^\mathbf{v} \cdot g_t^s)$ and a challenge $c \in \mathbb{Z}_p$, outputs a valid transcript $((\tilde{C}', c, \tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s}))$ without knowing γ , $\mathbf{v}$, or s .

1. Sample random values: $\tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p$, $\tilde{\mathbf{v}} \xleftarrow{\$} \mathbb{Z}_p^5$, $\tilde{s} \xleftarrow{\$} \mathbb{Z}_p$.
2. Compute the fake commitment $\tilde{C}'$ as: $\tilde{C}' = (C[0]^c g_2^{\tilde{\gamma}}, C[1]^c \circ \mathbf{A}^{\tilde{\mathbf{v}}}, C[2]^c \mathbf{b}^{\tilde{\mathbf{v}}} g_t^{\tilde{s}})$.
This ensures that the verification equation will pass with the response $(\tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s})$ and challenge c .
3. Output the simulated transcript: $((\tilde{C}', c, \tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s}))$.

In the real transcript, (γ, γ', s') are uniformly random in $\mathbb{Z}_p$, and $\mathbf{v}, \mathbf{v}'$ are uniformly random in $\mathbb{Z}_p^5$. Thus, $C' = (g_2^{\gamma'}, \mathbf{A}^{\mathbf{v}'}, \mathbf{b}^{\mathbf{v}'} \cdot g_t^{s'})$ is uniformly random. Since $c = H(C, C')$ is generated by the random oracle, $\hat{\gamma} = \gamma' - c\gamma$, $\hat{\mathbf{v}} = \mathbf{v}' - c\mathbf{v}$, $\hat{s} = s' - cs$ are uniform. In the simulation, $(\tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s})$ is randomly chosen from $\mathbb{Z}_p$ and distributed identically to $(\hat{\gamma}, \hat{\mathbf{v}}, \hat{s})$. $\tilde{C}'$ is also uniformly random since $(\tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s})$ is randomly chosen from $\mathbb{Z}_p$. Thus, the distribution of simulated transcript $((\tilde{C}', c, \tilde{\gamma}, \tilde{\mathbf{v}}, \tilde{s}))$ is identical to that of a real transcript. □

Theorem 8. *The CCATE instantiation II described in Figure 5 is **CCA2** secure against the game in Definition 5 in the generic group model and random oracle model.*

Proof : We also use a sequence of games starting with the real attack game and ending with a game where the adversary $\mathcal{A}$ has no advantage. Denote E_j as the event that $\mathcal{A}$ wins in each **Game_j**. Suppose the corrupted parties are the set $\mathbf{W}$, and the non-corrupted parties are the set $\mathbf{E}$.

- **Game₀**: This is the real attack game, where an adversary $\mathcal{A}$ can simulate an indistinguishability game (defined by Definition 5) as follows:
 1. *Setup*: The challenger first generates the CRS. For $i \in [1, n]$, C selects uniformly random values $u_i \leftarrow \mathbb{Z}_p$ as sk_i and sends $\text{pk}_i = (g_1^{u_i}, (g_1^{u_i L_i(\tau)}, g_1^{u_i(L_i(\tau) - L_i(0))}, g_1^{\frac{u_i(L_i^2(\tau) - L_i(\tau))}{Z(\tau)}}, g_1^{\frac{u_i(L_i(\tau) - L_i(0))}{\tau}}, \{g_1^{\frac{u_i L_i(\tau) L_j(\tau)}{Z(\tau)}}\}_{j \neq i}))$ to $\mathcal{A}$. Then, the challenger runs **VerifySetup** and sends (ek, ak) to $\mathcal{A}$.
 2. *Query*:
 - For $i \in \mathbf{W}$, the challenger sends sk_i to $\mathcal{A}$.
 - For polynomial times, $\mathcal{A}$ inquires a valid ciphertext C . The challenger verifies whether the ciphertext is valid. If not, it returns $\perp$; otherwise, it uses the extractor of the NIZK proof to obtain γ and responds $D_i = \text{pk}_{i,1}^\gamma$ for $i \in [1, n]$.
 3. Choosing s_0 and s_1 randomly, the challenger calculates a challenge ciphertext $C^* = (g_2^\gamma, \mathbf{A}^\mathbf{v}, \mathbf{b}^\mathbf{v} \cdot g_t^{s_0})$ and $\pi_{s_0}^* = (C'^*, c^*, \hat{\gamma}^*, \hat{\mathbf{v}}^*, \hat{s}^*)$ and sends (s_b, C^*) to $\mathcal{A}$, where $b \xleftarrow{\$} [0, 1]$.
 4. $\mathcal{A}$ makes additional queries, but is prohibited from making decryption queries on C^* .
 5. $\mathcal{A}$ makes a guess b' . If $b' = 0$, $\mathcal{A}$ guesses that C^* is generated from s_b , otherwise, it guesses C^* is not generated from s_b .
- **Game₁**: The game is identical to **Game₀**, except that the challenger simulates the proof $\pi_{s_0}^*$ in the challenge ciphertext without using the witness as described in Lemma 6. From the zero-knowledge property of the NIZK proof, $\Pr[E_1] = \Pr[E_0]$.
- **Game₂**: The game is identical to **Game₁**, except that the challenger responds to the decryption query as follows: For $i \in \mathbf{W}$, it returns $D_i = \text{pk}_{i,1}^\gamma$; For $i \in \mathbf{E}$, it returns random element. For $i \in \mathbf{E}$, the real partial decryption $D_i = C[0]^{\text{sk}_i}$ is a random element in the view of $\mathcal{A}$. Thus, the responses distribute identically to the decryption oracle in **Game₁** and $|\Pr[E_2] - \Pr[E_1]| \leq \text{negl}(\lambda)$.
- **Game₃**: This is the same as **Game₂**, except that we change the challenge ciphertext to the CPA-Secure STE ciphertext (remove the proof π^*) as in the CPA security game **Game_{CPA}** in [28] (i.e., the real attack game against CPA-Secure STE). Since π^* is simulated as random elements in **Game₂**, $|\Pr[E_3] - \Pr[E_2]| \leq \text{negl}(\lambda)$.

Therefore, $|\Pr[E_0] - \Pr[E_3]| \leq \text{negl}(\lambda)$. Since $\Pr[E_{\text{CPA}}] = \frac{1}{2} + \text{negl}(\lambda)$ was proved under generic group model in [28] and $|\Pr[E_3] - \Pr[E_{\text{CPA}}]| \leq \text{negl}(\lambda)$, we have $\Pr[E_3] = \frac{1}{2} + \text{negl}(\lambda)$. Thus, $\Pr[E_0] = \frac{1}{2} + \text{negl}(\lambda)$ and $\text{Adv}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}| \leq \text{negl}(\lambda)$.

6 Comparisons

6.1 Trade-offs in Setup

In elliptic curve-based PVSS schemes, exponentiation and bilinear pairing are the dominant computational operations. We denote E and P as the cost of an exponentiation and a bilinear pairing operation, respectively. For brevity, we use E to denote exponentiation operations across all elliptic curve groups—namely, $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$.

Setup	Implementation	Assumption	Reconf.	Aggregator	Comp.	Comm.
Figure 2	Committee-based DKG	1-honest committee & t -honest parties	✓	$\perp$	$O(t \cdot N)E$	$O(t \cdot N) \mathbb{G} + O(N) \mathbb{Z}_p $
Figure 4	Power-of-Tau & Silent Setup DKG	Honest parties	✓	Honest-but-curious	$O(n^2)E$	$O(n)E$

Table 1. Comparison of setup between the proposed two constructions

Previous PVSS schemes assume a Public Key Infrastructure (PKI) and typically require minimal communication and computational overhead during setup. In contrast, our scheme reduces the complexity of distribution and verification to constant levels, albeit at the cost of increased setup complexity. Table 1 compares the setup procedures for constructing CCATE and PVSS schemes. The setup in Figure 2 leverages an external N -size committee to execute DKG for n parties, as long as at least one committee member is honest. The DKG requires each committee member to invoke Feldman VSS scheme, regarding each party as a shareholder. Each party requires $O(t)$ exponentiations to check the correctness of his share against commitment of coefficients, i.e., $\{g^{a_i}\}_{i \in [0, t-1]}$. Hence, the computation complexity is $O(t \cdot N)E$ and communication complexity is $O(t \cdot N)|\mathbb{G}| + O(N)|\mathbb{Z}_p|$ for each party. We also introduce the functionality to change parties, where the DKG process is applied once again. Figure 4 illustrates the silent setup DKG protocol, which enables each party to independently sample its secret/public key pair without any interaction with other parties. Each honest party must generate and publish $O(n)$ group elements (i.e., $h_{i,5,j}$), where the computation of each element requires $O(n)$ exponentiations due to evaluation of polynomial on $\mathbb{G}$ (refer to Appendix C). This results in a total computational cost of $O(n^2)$ exponentiations per party. However, this approach relies on a pre-computed Power-of-Tau CRS and requires an honest-but-curious aggregator to validate and aggregate public keys, producing aggregated verification structures.

Table 1 presents a clear efficiency trade-off between committee-based DKG and silent setup DKG. The former necessitates interactive communication among participants, resulting in substantial bandwidth overhead that scales with the number of parties. In contrast, the silent setup eliminates all inter-party interaction, shifting the burden to local computation — each party must perform a total of $O(n^2)$ exponentiations.

6.2 Comparison with State-of-The-Art PVSS

Table 2 compares state-of-the-art PVSS schemes in terms of computational and communication complexity. Since all schemes use Lagrange interpolation for reconstruction once t valid shares are obtained, the reconstruction phase is similar across approaches. Therefore, we primarily focus on the technique analysis and complexity comparison in distribution and verification phases.

In Figure 3, each ElGamal-style ciphertext, i.e., C and C' , incurs a computational cost of $3E$ in the **Encrypt** algorithm. Consequently, the total encryption cost is $6E$. The size of the output (C, π_s) is $4|\mathbb{G}| + 3|\mathbb{Z}_p|$. In the **VerifyCT** algorithm, the total verification cost amounts to $5E$: verifying $C[0]$ requires $3E$, while verifying $C[1]$ requires $2E$. In Figure 5, each non-zero element in matrix scalar exponentiation requires $1E$, leading to a total of $38E$ in the calculation of C and C' in the **Encrypt** algorithm. Since $C[0]$ and $C[2]$ are single element and $C[1]$ is a tuple containing 8 group elements. Thus, the **VerifyCT** algorithm requires $25E$. The **Decrypt** algorithm requires $O(n)E$, because w requires calculation of $Q(x) = g_1^{Q_x(\tau)}$ (refer to the details in Figure 12), where $Q_x(X)$ is a n -degree polynomial.

Ref.	Dist. Comp.	Dist. Comm.	Verif. Comp	Recon. Comp.	Recon. Comm.
SCRAPE _{DDH} [7]	$O(n)E$	$O(n)(\mathbb{G} + \mathbb{Z}_p)$	$O(n)E$	$O(n)E$	$O(t)(\mathbb{G} + \mathbb{Z}_p)$
SCRAPE _{DBS} [7]	$O(n)E$	$O(n) \mathbb{G} $	$O(n)E + O(n)P$	$O(t)E + O(t)P$	$O(t) \mathbb{G} $
ALBATROSS [8]	$O(n)E$	$O(n)(\mathbb{G} + \mathbb{Z}_p)$	$O(n)E$	$O(t)E$	$O(t)(\mathbb{G} + \mathbb{Z}_p)$
HEPVSS [9]	$O(n)E$	$O(n)(\mathbb{G} + \mathbb{Z}_p)$	$O(n)E$	$O(t)E$	$O(t) \mathbb{G} $
DHPVSS [9]	$O(n)E^\dagger$	$O(n) \mathbb{G} $	$O(n)E^\dagger$	$O(t)E$	$O(t)(\mathbb{G} + \mathbb{Z}_p)$
AsyRand [18]	$O(n)E$	$O(n)(\mathbb{G} + \mathbb{Z}_p)$	$O(n)E$	$O(t)E + O(t)P$	$O(t) \mathbb{G} $
Figure 3 (Pairing-free)	$O(1)E$ i.e., $6E$	$O(1)$ i.e., $4 \mathbb{G} + 3 \mathbb{Z}_p $	$O(1)E$ i.e., $5E$	$O(t)E$	$O(t)(\mathbb{G} + \mathbb{Z}_p)$
Figure 5 (Silent setup)	$O(1)E$ i.e., $38E$	$O(1)$ i.e., $20 \mathbb{G} + 8 \mathbb{Z}_p $	$O(1)E$ i.e., $25E$	$O(n)E$	$O(t)(\mathbb{G} + \mathbb{Z}_p)$

[†]: As DHPVSS requires n times of $(n - t - 1)$ -polynomial evaluation in the sharing phase, making it require a bit more than $O(n)$ exponentiations.

Table 2. PVSS comparison

SCRAPE [7] is the first PVSS scheme to achieve $O(n)$ verification complexity. In the sharing phase, the dealer selects a random secret s and constructs a polynomial of degree at most $t - 1$, with s as the constant term. Each party receives a share based on evaluating this polynomial at their index. The dealer encrypts each share using the party's public key and also computes a commitment to the share using generator g . In SCRAPE_{DDH}, a Discrete Log Equality (DLEQ) proof is generated for each encrypted share to demonstrate its correctness. Any external verifier can check these DLEQ proofs to confirm that the encrypted shares are consistent with valid secret exponents. In contrast, in

SCRAPE_{DBS}, verification is performed using a pairing-based check. If the verification of encrypted shares pass, the verifier samples a random codeword from the dual code of the secret sharing scheme and verifies a multiplicative condition across all shares. This check ensures that the shared values are consistent with the dealer’s polynomial.

In ALBATROSS [8], the dealer selects a random polynomial of degree at most $t + \ell - 1$, where ℓ is the number of secrets to be shared. The secret values $S_0, S_1, \dots, S_{\ell-1}$ are derived from the polynomial evaluated at specific points and then exponentiated with the generator h . For each participant P_i , the dealer not only generates the encrypted share but also a proof called LDEI (Logarithmic Discrete Exponentiation Integrity). The LDEI proof ensures that the encrypted shares are consistent with a valid polynomial of limited degree. For verification, any external verifier checks the LDEI proof to confirm that the published shares are generated correctly and correspond to a legitimate polynomial.

In HEPVSS [9], the dealer creates a polynomial based on the secret and computes shares using Shamir’s method. Each share is encrypted using ElGamal under the recipient’s public key. The dealer also produces a non-interactive proof that links the encrypted shares to a valid polynomial, ensuring correctness. Further in DHPVSS [9], the shares are then encrypted using the dealer’s private key and the parties’ public keys to ensure confidentiality and authenticity. To enable public verification, the dealer creates a commitment to the polynomial using a hash function. This commitment helps bind the distribution to the correct secret without revealing it. The dealer then computes two combined values, U and V , by taking weighted sums of encrypted shares and public key components. The weights come from reconstruction coefficients, ensuring that only valid combinations contribute correctly. A DLEQ proof for (U, V) is generated to enable the dealer to publicly prove knowledge of the private key used in encryption. The algorithm still requires linear time in the number of participants for both sharing and verification. It uses only basic group operations and avoids bilinear pairing computations.

The experimental results, compared with DHPVSS [9], are presented in Appendix E.

7 Conclusion

In this paper, we have introduced a novel paradigm for constructing Publicly Verifiable Secret Sharing (PVSS) schemes with $O(1)$ complexity in the critical share distribution and verification phases. Our approach is built upon a CCA2-Secure Threshold Encryption (CCATE), which incorporates NIZK proofs to achieve strong public verifiability. We have presented a generic framework that transforms any CCATE scheme into a secure PVSS protocol, demonstrating that the problem of scalable secret sharing can be reduced to the construction of an efficient threshold encryption scheme.

To instantiate our framework, we provided two distinct constructions, highlighting its flexibility and practicality. The first construction is pairing-free and

based on a committee-driven Distributed Key Generation (DKG) protocol combined with Threshold ElGamal encryption, offering high efficiency per party during setup. The second construction leverages a silent setup paradigm using a Power-of-Tau ceremony, enabling non-interactive key generation ideal for open, permissionless environments. Both constructions achieve the desired constant-time complexity for distribution and verification, marking a significant improvement over existing $O(n)$ schemes.

Our work establishes a powerful connection between threshold encryption and PVSS, paving the way for future advancements. Promising directions for future research include exploring alternative instantiations of the CCATE primitive under different cryptographic assumptions, including post-quantum candidates, to enhance long-term security. Furthermore, the practical implications of our PVSS schemes are significant for real-world systems. A direct application is in DRB protocols, where our $O(1)$ -complexity PVSS provides a solution with minimal overhead during the commitment phase. Additionally, the support for dynamic PVSS membership updates is particularly transformative for PoS blockchain consensus protocols, as it directly bridges a fundamental gap between permissioned and permissionless systems.

References

1. Stadler, M. Publicly verifiable secret sharing. In *Eurocrypt*, 1996, pp. 190-199.
2. Schoenmakers, B. A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic Voting. In *CRYPTO*, 1999, pp. 148-164.
3. Kokoris-Kogias, E., Alp, E.C., Gasser, L., Jovanovic, P., Syta, E. and Ford, B. Calypso: Private data management for decentralized ledgers. *Proc. VLDB Endow.*, 2020, pp. 586-599.
4. Bagheri, K., Knapen, N., Nicolas, G. and Rahimi, M. Pre-constructed publicly verifiable secret sharing and applications. In *ACNS*, 2025, pp. 89-119.
5. Zyskind, G., Yanai, A. and Pentland, A. Unstoppable wallets: Chain-assisted threshold ECDSA and its applications. In *Asia CCS*, 2024, pp. 1844-1860.
6. Saereesitthipitak, S. and Zindros, D. Cassiopeia: Practical On-Chain Witness Encryption. In *FC*, 2023, pp. 385-404.
7. Cascudo, I. and David, B. SCRAPE: Scalable Randomness Attested by Public Entities. In *ACNS*, 2017, pp. 537-556.
8. Cascudo, I. and David, B. ALBATROSS: Publicly Attestable Batched Randomness Based on Secret Sharing. In *ASIACRYPT*, 2020, pp. 311-341.
9. Cascudo, I., David, B., Garms, L. and Konring, A. YOLO YOSO: Fast and Simple Encryption and Secret Sharing in the YOSO Model. In *ASIACRYPT*, 2022, pp. 651-680.
10. Cascudo, I. and David, B. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In *Eurocrypt*, 2024, pp. 216-248.
11. Damgård, I. Commitment schemes and zero-knowledge protocols. In School organized by the European Educational Forum, 1998, pp. 63-86.
12. Franklin, M. and Yung, M. Communication complexity of secure computation. In *STOC*, 1992, pp. 699-710.
13. Das, S., Krishnan, V., Isaac, I. M. and Ren, L. Spurt: Scalable distributed randomness beacon with transparent setup. In *S&P*, 2022, pp. 2502-2517.

14. Bhat, A., Shrestha, N., Kate, A. and Nayak, K. OptRand: Optimistically Responsive Reconfigurable Distributed Randomness. In *NDSS*, 2023.
15. Bacho, R., Lenzen, C., Loss, J., Ochsenreither, S. and Papachristoudis, D. GRandom-Line: adaptively secure DKG and randomness beacon with (log-)quadratic communication complexity. In *CCS*, 2024, pp. 941-955.
16. Kiayias, A., Russell, A., David, B. and Oliynykov, R. Ouroboros: A provably secure proof-of-stake blockchain protocol. In *CRYPTO*, 2017, pp. 357-388.
17. Ballweg, J., Cai, Z. and Goharshady, A.K., 2023, December. PureLottery: Fair leader election without decentralized random number generation. In *ICBC*, 2023, pp. 273-280.
18. Zhang, L., Liu, T., Kan, H. and Zhang, J. AsyRand: asynchronous distributed randomness beacon with reconfiguration. *Cryptology ePrint Archive*, 2025.
19. Gurkan, K., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G. and Tomescu, A. Aggregatable distributed key generation. In *Eurocrypt*, 2024, pp. 147-176.
20. Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L. and Ren, L. Practical asynchronous distributed key generation. In *S&P*, 2022, pp. 2518-2534.
21. Damgård, I. On Σ -Protocols, <https://www.cs.au.dk/~ivan/Sigma.pdf>, Accessed: 2025-09-16
22. Fiat, A. and Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In *Eurocrypt*, 2025, pp. 186-194.
23. Libert, B. and Yung, M. Non-interactive CCA-secure threshold cryptosystems with adaptive security: New framework and constructions. In *CRYPTO*, 2012, pp. 75-93.
24. Chaum, D. and Pedersen, T.P. Wallet databases with observers. In *CRYPTO*, 1992, pp. 89-105.
25. Groneberg, B. Threshold Cryptography. Technische Berichte des Hasso-Plattner-Instituts für Softwaresystemtechnik an der Universität Potsdam, p.5.
26. Nikolaenko, V., Ragsdale, S., Bonneau, J. and Boneh, D. Powers-of-tau to the people: Decentralizing setup ceremonies. In *ACNS*, 2024, pp. 105-134.
27. Garg, S., Jain, A., Mukherjee, P., Sinha, R., Wang, M. and Zhang, Y. hints: Threshold signatures with silent setup. In *S&P*, 2024, pp. 3034-3052.
28. Garg, S., Kolonelos, D., Policharla, G.V. and Wang, M. Threshold encryption with silent setup. In *CRYPTO*, 2024, pp. 352-386.
29. Goyal, V., Kothapalli, A., Masserova, E., Parno, B. and Song, Y. Storing and retrieving secrets on a blockchain. In *PKC*, 2022, pp. 252-282.
30. Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T. and Reyzin, L. Can a public blockchain keep a secret?. In *TCC*, 2020, pp. 260-290.
31. Choudhuri, A.R., Garg, S., Piet, J. and Policharla, G.V. Mempool privacy via batched threshold encryption: Attacks and defenses. In *USENIX Security*, 2024, pp. 3513-3529.
32. Choudhuri, A.R., Garg, S., Policharla, G.V. and Wang, M. Practical mempool privacy via one-time setup batched threshold encryption. In *USENIX Security*, 2025, pp. 3477-3495.
33. Katz, J. and Lindell, Y. Introduction to modern cryptography: principles and protocols. Chapman and hall/CRC. 2007.
34. Fujisaki, E. and Okamoto, T. Secure integration of asymmetric and symmetric encryption schemes. In *CRYPTO*, 1999, pp. 537-554.
35. Cramer, R. and Shoup, V. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In *CRYPTO*, 1998, pp. 13-25.
36. Naor, M. and Yung, M. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *STOC*, 1990, pp. 427-437.

- 37. Feldman, P. A practical scheme for non-interactive verifiable secret sharing. In *SFCS*, 1987, pp. 427-438.
- 38. Zhang, L., Wu, X., Ma, Y. and Kan, H. Data exchange for the metaverse with accountable decentralized TTPs and incentive mechanisms. *IEEE TBD*, 2025, 11(5): 2431-2442.
- 39. Ràfols, C. and Zapico, A. An algebraic framework for universal and updatable SNARKs. In *CRYPTO*, 2021, pp. 774-804.
- 40. Ostrovsky, R. and Yung, M. How to withstand mobile virus attacks. In *PODC*, 1991, pp. 51-59.
- 41. Groth, J. Non-interactive distributed key generation and key resharing. *Cryptology ePrint Archive*, 2021.

A Bilinear Mapping

Let $\mathbb{G}_1, \mathbb{G}_2$ be additive groups and $\mathbb{G}_T$ a multiplicative group, all of prime order p . A bilinear map is a function $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, satisfying:

- **Bilinearity:** $\forall g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2, a, b \in \mathbb{Z}_p$, we have $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$.
- **Non-degeneracy:** $\exists g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ such that $e(g_1, g_2) \neq 1_{\mathbb{G}_T}$.
- **Computability:** e is efficiently computable.

If vector $A = (a_1, a_2, \dots, a_n)$ and vector $B = (b_1, b_2, \dots, b_n)$, we define $e(A, B) = \prod_{i=1}^n e(a_i, b_i)$.

B Feldman VSS

Feldman VSS scheme [37,38] enables the dealer to be privately verifiable.

- $(\mathbb{G}, p, g) \leftarrow \text{VSS.Setup}(1^\lambda)$: g is a generator of group $\mathbb{G}$ with prime order p .
- $(i, s_i, \{gs_i\}_{i \in [1, n]}, \{Com_j\}_{j \in [0, t-1]}) \leftarrow \text{VSS.Share}(s, n, t)$: Choose a polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i \pmod{p}$, compute $s_i = f(i)$ and $Com_j = g^{a_j} \pmod{p}$. The dealer broadcasts $(\{gs_i = g^{s_i}\}, \{Com_j\})$ and sends (i, s_i) securely to the shareholder i .
- $\text{true or false} \leftarrow \text{VSS.Verify}(i, s_i, \{gs_i\}, \{Com_j\})$: Each shareholder tests whether $gs_i \stackrel{?}{=} g^{s_i} \stackrel{?}{=} \prod_{j=0}^{t-1} (Com_j)^{i^j} \pmod{p}$ holds. If not, the shareholder broadcasts a complaint against the dealer. Note that if a dealer is malicious, it needs to publish the received s_i in the complaint phase.
- $s \leftarrow \text{VSS.Reconstruct}(\{(j, s_j)\}_{j \in Q})$: With $|Q| \geq t$, it recovers s using the Lagrange interpolation algorithm `Intpl`.

C Polynomials over $\mathbb{Z}_p$

Vanishing Polynomial Let $\omega \in \mathbb{Z}_p$ be an ℓ -th primitive root of unity, i.e. $\omega^\ell = 1$ over $\mathbb{Z}_p$. ω generates the multiplicative subgroup of roots of unity $\mathbb{H} = \{\omega, \omega^2, \dots, \omega^\ell\}$. Let $Z(x) = \prod_{i=1}^\ell (x - \omega^i)$ be the vanishing polynomial on $\mathbb{H}$. Since $\mathbb{H}$ is a multiplicative subgroup, $Z(x) = x^\ell - 1$ and $L_i(x) = \frac{\omega^i}{\ell} \cdot \frac{x^\ell - 1}{x - \omega^i}$. Note that $L_i(0) = \frac{1}{\ell}$. Sometimes, we will refer to ω^ℓ as ω_0 and $L_0 = L_\ell$, as $\mathbb{H}$ is cyclic.

Evaluation of Polynomial on $\mathbb{G}$ Let $\tau \xleftarrow{\$} \mathbb{Z}_p$ be the unknown secret trapdoor. Given a polynomial $f(x) = \sum_{i=0}^d a_i x^i$, the evaluation of polynomial on $\mathbb{G}$ at τ is defined and calculated as: $g^{f(\tau)} := g^{\sum_{i=0}^d a_i \tau^i} = \prod_{i=0}^d (g^{\tau^i})^{a_i}$, where g^{τ^i} are publicly known. Apparently, this evaluation requires $d + 1$ exponentiations.

D Discrete Logarithm Equality (DLEQ)

The Chaum-Pedersen Protocol is a sigma protocol for discrete logarithm equality (DLEQ) [24]. Let $\mathbb{G}$ be a group with DL assumption and g, G, t, T be group elements. The DLEQ relation is $\mathcal{R}_{DLEQ} = \{(g, G, t, T), s \mid G = g^s \wedge T = t^s\}$ and runs as follows:

Functionality Non-interactive DLEQ

$\pi_s \leftarrow \underline{DLEQ(g, G, t, T, s)}$

$r \xleftarrow{\$} \mathbb{Z}_p$

$G' = g^r, T' = t^r$

$c = H(G, T, G', T')$

$z = r + c \cdot s$

$\pi_s = (G', T', c, z)$

$\text{bool} \leftarrow \underline{DLEQVrf(g, G, t, T, \pi_s)}$

return $g^z \stackrel{?}{=} G'G^c$ and $t^z = T'T^c$

Fig. 6. DLEQ proof generation and verification

E Experimental Results

We implement the proposed PVSS schemes in Rust using the arkworks library⁵. All benchmarks are conducted on a 2024 MacBook Pro equipped with an M4 Max chip, a 16-core CPU, and 48 GB of RAM, running in single-threaded mode. We employ the BLS12-381 elliptic curve and set the threshold to $t = n/2 + 1$ for all experiments, where n denotes the number of parties. To highlight our strengths, we compare the proposed PVSS schemes with DHPVSS [9], the most recent state-of-the-art construction. In all experiments, we precompute the Lagrange coefficients for each PVSS scheme to ensure fair performance evaluation. The computational costs of the different algorithms are illustrated in Figures 7 to 11.

⁵ <https://github.com/arkworks-rs>

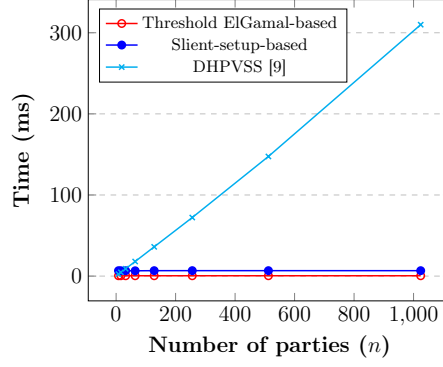


Fig. 7. Cost of PVSS.Share

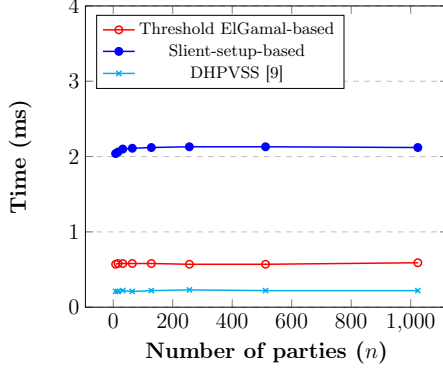


Fig. 9. Cost of PVSS.PreRecon

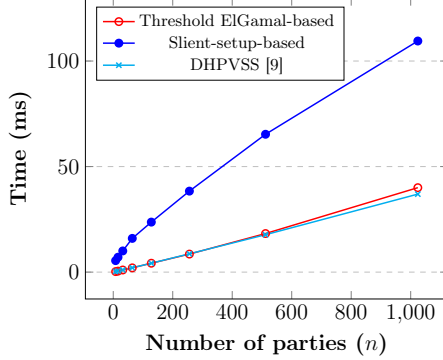


Fig. 11. Cost of PVSS.Recon

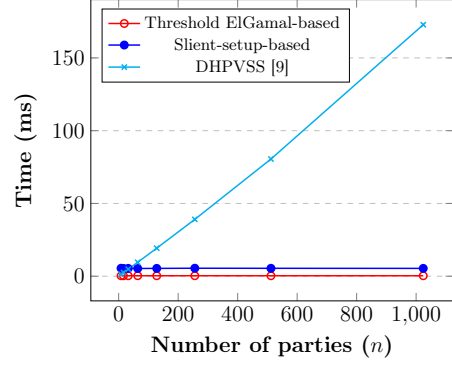


Fig. 8. Cost of PVSS.Verify

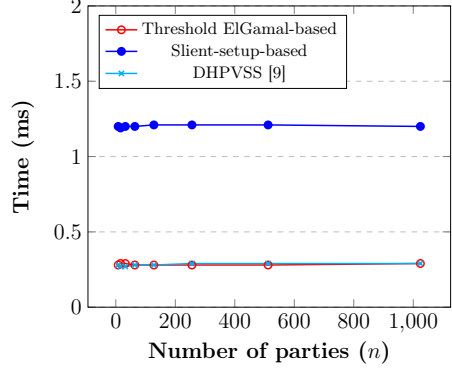


Fig. 10. Cost of PVSS.DecShareVrf

F CPA-Secure Silent Setup Threshold Encryption [28]

Lemma 7 (Univariate Sumcheck[39]). *Let $A(x) = \sum_{i=0}^n a_i L_i(x)$ and $B(x) = \sum_{i=0}^n b_i L_i(x)$, Then:*

$$A(x)B(x) = \frac{\sum_i a_i b_i}{n} + Q_z(x)Z(x) + Q_x(x)x$$

holds, where $Q_x(x) = \sum_{i=0}^n a_i b_i \frac{L_i(x) - L_i(0)}{x}$ and $Q_z(x) = \sum_{i=0}^n a_i b_i \frac{L_i^2(x) - L_i(x)}{Z(x)} + \sum_{j=1, j \neq i}^n a_i b_j \frac{L_i(x)L_j(x)}{Z(x)}$.

Figure 12 demonstrates the CPA-secure STE concrete construction. The construction builds on a univariate sumcheck protocol (Lemma 7) and a silent setup threshold signature (STS) scheme. It has been shown that a STE scheme can be constructed from any signature scheme whose verification process forms a public linear constraint system. Consequently, the core of the STE scheme lies in finding a matrix $\mathbf{A}$ that encapsulates the verification logic of a threshold signature. The encryption key in the STE scheme corresponds to the verification key of STS, while decryption involves aggregating a valid signature that satisfies STS verification. Specifically, the matrix $\mathbf{A}$ in STE encryption serves as the STS verification key, and the decryption vector $\mathbf{w}$ represents a valid aggregated STS signature, augmented with elements derived from the univariate sumcheck protocol used during signature verification.

Though, [28] presents a method to achieve CCA-security: a tag-based NIZK proof which proves knowledge of γ . However, this method does not guarantee well-formed ciphertext, thus unsuitable for our CCATE scheme.

Functionality CPA-Secure Threshold Encryption via Silent Setup

$(\text{pp}, \{\text{sk}_i, \text{pk}_i\}) \leftarrow \mathbf{Setup}(\lambda, t, n)$

As introduced in Figure 4

$\text{bool} \leftarrow \mathbf{VerifySetup}(\text{pp}, \{\text{pk}_i\})$

As introduced in Figure 4

$(C) \leftarrow \mathbf{Encrypt}(s, \text{pp})$

Sample $\gamma \xleftarrow{\$} \mathbb{Z}_p$, set $Z_0 = g_2^{\tau-1}$, set $\mathbf{b} = (1_{\mathbb{G}_T}, 1_{\mathbb{G}_T}, 1_{\mathbb{G}_T}, 1_{\mathbb{G}_T}, g_t)^\top$, and set $\mathbf{A}$ (the same as in Figure 5)

Sample $s_1, \dots, s_5 \xleftarrow{\$} \mathbb{Z}_p^*$ and set $\mathbf{v} = (s_1, \dots, s_5)^\top$

$C = (g_2^\gamma, \mathbf{A}^\mathbf{v}, \mathbf{b}^\mathbf{v} \cdot g_t^s)$ ($\mathbf{A}^\mathbf{v}, \mathbf{b}^\mathbf{v}$ denotes matrix scalar exponentiation)

$D_i \leftarrow \mathbf{PartialDec}(C, \text{sk}_i)$

$D_i = C[0]^{\text{sk}_i} = g_2^{\gamma \text{sk}_i}$

$\text{bool} \leftarrow \mathbf{VerifyPD}(D_i, \text{pk}_i, C)$

$e(\text{pk}_i, g_2^\gamma) \stackrel{?}{=} e(g_1, D_i)$

$S \leftarrow \mathbf{Decrypt}(C, \{D_i\}_{i \in P})$

- Using ak to compute the subset with valid public keys in P , i.e., $P_I = P \cap I$. Interpolate a polynomial $B(X)$ such that $B(\omega^0) = 1, B(\omega^i) = 0$ for $i \notin P_I$ and set $B = g_2^{\sum_{i=0}^n B(\omega^i) L_i(\tau)}$.
- Compute $\text{pk}' = (g_1 \prod_{i \in P_I} \text{pk}_i^{B(\omega^i)})^{\frac{1}{n+1}}$ as aggregated public key.
- Compute polynomial $Q_x(X)$ and $Q_z(X)$ such that $\mathbf{SK}(X)B(X) = \frac{\text{sk}'}{n+1} + Q_z(X)Z(X) + Q_x(X)X$ where $\mathbf{SK}(X) = \sum_{i=1}^n \text{sk}_i L_i(X)$ and $\text{sk}' = \sum_{i=0}^n \mathbf{SK}(\omega^i)B(\omega^i) = \sum_{i \in P_I} \text{sk}_i B(\omega^i) + 1$. According to Lemma 7 univariate sumcheck: $Q_x(X) = \sum_{i=0}^n B(\omega^i)(\text{sk}_i \frac{L_i(X) - L_i(0)}{X})$, $Q_z(X) = \sum_{i=0}^n B(\omega^i)(\text{sk}_i \frac{L_i^2(X) - L_i(X)}{Z(X)}) + \sum_{i=0}^n B(\omega^i) \sum_{j=0, j \neq i}^n (\text{sk}_j \frac{L_i(X)L_j(X)}{Z(X)})$. Compute $Q_z = g_1^{Q_z(\tau)}$.
- Compute $Q(x) = g_1^{Q_x(\tau)}$ and $\hat{Q}(x) = g_1^{Q_x(\tau) \cdot \tau}$.
- Set $D = \frac{g_2^{\sum_{i \in P_I} D_i^{B(\omega^i)}}}{n+1}$ as aggregated partial decryption.
- Compute $\hat{B} = g_1^{\tau^t B(\tau)}$.
- Compute $Q_0(X)$ such that $B(X) - 1 = Q_0(X)(X - \omega^0)$ and its commitment $Q_0 = g_1^{Q_0(\tau)}$.
- $\mathbf{w}$ is set as $(B, -\text{pk}', -Q_z, -Q_x, \hat{Q}_x, D, -\hat{B}, -Q_0)$.
- Output $S = \frac{C[2]}{e(C[1], \mathbf{w})} = \frac{\mathbf{b}^\mathbf{v} \cdot g_t^s}{e(\mathbf{A}^\mathbf{v}, \mathbf{w})} = \frac{\mathbf{b}^\mathbf{v} \cdot g_t^s}{e(\mathbf{A}, \mathbf{w})^\mathbf{v}} = \frac{\mathbf{b}^\mathbf{v} \cdot g_t^s}{\mathbf{b}^\mathbf{v}} = g_t^s$.

Fig. 12. CPA-Secure Threshold Encryption via Silent Setup